Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com/. Reproduction and distribution is prohibited.

PD ISO/IEC TS 17961:2013

Incorporating corrigendum August 2016

BSI Standards Publication

Information technology — Programming languages, their environments and system software interfaces — C secure coding rules

bsi.

National foreword

This Published Document is the UK implementation of ISO/IEC TS 17961:2013, incorporating corrigendum August 2016.

The start and finish of text introduced or altered by corrigendum is indicated in the text by tags. Text altered by ISO/IEC corrigendum August 2016 is indicated in the text by AC1 AC1.

The UK participation in its preparation was entrusted to Technical Committee IST/5, Programming languages, their environments and system software interfaces.

A list of organizations represented on this committee can be obtained on request to its secretary.

This publication does not purport to include all the necessary provisions of a contract. Users are responsible for its correct application.

© The British Standards Institution 2016.
Published by BSI Standards Limited 2016

ISBN 978 0 580 95600 3

ICS 35.060

Compliance with a British Standard cannot confer immunity from legal obligations.

This British Standard was published under the authority of the Standards Policy and Strategy Committee on 30 November 2013.

Amendments/corrigenda issued since publication

Date	Text affected
30 September 2016	Implementation of ISO/IEC corrigendum August 2016

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

PD ISO/IEC TS 17961:2013

TECHNICAL SPECIFICATION

ISO/IEC TS 17961

First edition
2013-11-15

Information technology — Programming languages, their environments and system software interfaces — C secure coding rules

Technologies de l'information — Langages de programmation, leur environnement et interfaces des logiciels de systèmes — Règles de programmation sécurisée en C

Reference number
ISO/IEC TS 17961:2013(E)

© ISO/IEC 2013

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

COPYRIGHT PROTECTED DOCUMENT

© ISO/IEC 2013

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from either ISO at the address below or ISO's member body in the country of the requester.

ISO copyright office
Case postale 56 • CH-1211 Geneva 20
Tel. + 41 22 749 01 11
Fax + 41 22 749 09 47
E-mail copyright@iso.org
Web www.iso.org

Published in Switzerland

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Contents

Page

Foreword ..... **v**

Introduction ..... **vi**

1 Scope **1**

2 Conformance **1**
2.1 Portability assumptions 2

3 Normative references **2**

4 Terms and definitions **2**

5 Rules **5**
5.1 Accessing an object through a pointer to an incompatible type [ptrcomp] 5
5.2 Accessing freed memory [accfree] 6
5.3 Accessing shared objects in signal handlers [accsig] 7
5.4 No assignment in conditional expressions [boolasgn] 8
5.5 Calling functions in the C Standard Library other than `abort`, `_Exit`, and `signal` from within a signal handler [asyncsig] 9
5.6 Calling functions with incorrect arguments [argcomp] 11
5.7 Calling `signal` from interruptible signal handlers [sigcall] 12
5.8 Calling `system` [syscall] 13
5.9 Comparison of padding data [padcomp] 14
5.10 Converting a pointer to integer or integer to pointer [intptrconv] 14
5.11 Converting pointer values to more strictly aligned pointer types [alignconv] 15
5.12 Copying a `FILE` object [filecpy] 16
5.13 Declaring the same function or object in incompatible ways [funcdecl] 16
5.14 Dereferencing an out-of-domain pointer [nullref] 18
5.15 Escaping of the address of an automatic object [addrescape] 18
5.16 Conversion of signed characters to wider integer types before a check for `EOF` [signconv] 19
5.17 Use of an implied default in a `switch` statement [swtchdflt] 19
5.18 Failing to close files or free dynamic memory when they are no longer needed [fileclose] 20
5.19 Failing to detect and handle standard library errors [liberr] 20
5.20 Forming invalid pointers by library function [libptr] 26
5.21 Allocating insufficient memory [insufmem] 28
5.22 Forming or using out-of-bounds pointers or array subscripts [invptr] 29
5.23 Freeing memory multiple times [dblfree] 34
5.24 Including tainted or out-of-domain input in a format string [usrfmt] 35
5.25 Incorrectly setting and using `errno` [inverrno] 37
5.26 Integer division errors [diverr] 39
5.27 Interleaving stream inputs and outputs without a flush or positioning call [ioileave] 40
5.28 Modifying string literals [strmod] 41
5.29 Modifying the string returned by `getenv`, `localeconv`, `setlocale`, and `strerror` [libmod] 42
5.30 Overflowing signed integers [intoflow] 43
5.31 Passing a non-null-terminated character sequence to a library function that expects a string [nonnullcs] 44
5.32 Passing arguments to character-handling functions that are not representable as `unsigned char` [chrsgnext] 45
5.33 Passing pointers into the same object as arguments to different `restrict`-qualified parameters [restrict] 46
5.34 Reallocating or freeing memory that was not dynamically allocated [xfree] 47
5.35 Referencing uninitialized memory [uninitref] 48
5.36 Subtracting or comparing two pointers that do not refer to the same array [ptrobj] 49
5.37 Tainted strings are passed to a string copying function [taintstrcpy] 50

© ISO/IEC 2013 – All rights reserved

5.38 Taking the size of a pointer to determine the size of the pointed-to type [sizeofptr] 50
5.39 Using a tainted value as an argument to an unprototyped function pointer [taintnoproto] 51
5.40 Using a tainted value to write to an object using a formatted input or output function [taintformatio] 52
5.41 Using a value for `fsetpos` other than a value returned from `fgetpos` [xfilepos] 52
5.42 Using an object overwritten by `getenv`, `localeconv`, `setlocale`, and `strerror` [libuse] 53
5.43 Using character values that are indistinguishable from `EOF` [chreof] 54
5.44 Using identifiers that are reserved for the implementation [resident] 55
5.45 Using invalid format strings [invfmtstr] 57
5.46 Tainted, potentially mutilated, or out-of-domain integer values are used in a restricted sink [taintsink] 58

Annex A (informative) **Intra- to Interprocedural Transformations** **59**

Annex B (informative) **Undefined Behavior** **63**

Annex C (informative) **Related Guidelines and References** **71**

Annex D (informative) **Decidability of Rules** **77**

Bibliography **78**

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Foreword

ISO (the International Organization for Standardization) and IEC (the International Electrotechnical Commission) form the specialized system for worldwide standardization. National bodies that are members of ISO or IEC participate in the development of International Standards through technical committees established by the respective organization to deal with particular fields of technical activity. ISO and IEC technical committees collaborate in fields of mutual interest. Other international organizations, governmental and non-governmental, in liaison with ISO and IEC, also take part in the work. In the field of information technology, ISO and IEC have established a joint technical committee, ISO/IEC JTC 1.

International Standards are drafted in accordance with the rules given in the ISO/IEC Directives, Part 2.

The main task of the joint technical committee is to prepare International Standards. Draft International Standards adopted by the joint technical committee are circulated to national bodies for voting. Publication as an International Standard requires approval by at least 75 % of the national bodies casting a vote.

In other circumstances, particularly when there is an urgent market requirement for such documents, the joint technical committee may decide to publish an ISO/IEC Technical Specification (ISO/IEC TS), which represents an agreement between the members of the joint technical committee and is accepted for publication if it is approved by 2/3 of the members of the committee casting a vote.

An ISO/IEC TS is reviewed after three years in order to decide whether it will be confirmed for a further three years, revised to become an International Standard, or withdrawn. If the ISO/IEC TS is confirmed, it is reviewed again after a further three years, at which time it must either be transformed into an International Standard or be withdrawn.

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. ISO shall not be held responsible for identifying any or all such patent rights.

ISO/IEC TS 17961 was prepared by Joint Technical Committee ISO/IEC JTC 1, *Information technology*, Subcommittee SC 22, *Programming languages, their environments and system software interfaces*.

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Introduction

Background

An essential element of secure coding in the C programming language is a set of well-documented and enforceable coding rules. The rules specified in this Technical Specification apply to analyzers, including static analysis tools and C language compiler vendors that wish to diagnose insecure code beyond the requirements of the language standard. All rules are meant to be enforceable by static analysis.

The application of static analysis to security has been done in an ad hoc manner by different vendors, resulting in nonuniform coverage of significant security issues. This specification enumerates secure coding rules and requires analysis engines to diagnose violations of these rules as a matter of conformance to this specification. These rules may be extended in an implementation-dependent manner, which provides a minimum coverage guarantee to customers of any and all conforming static analysis implementations.

The largest underserved market in security is ordinary, non-security-critical code. The security-critical nature of code depends on its purpose rather than its environment. The UNIX finger daemon (`fingerd`) is an example of ordinary code, even though it may be deployed in a hostile environment. A user runs the client program, `finger`, which sends a user name to `fingerd` over the network, which then sends a reply indicating whether the user is logged in and a few other pieces of information. The function of `fingerd` has nothing to do with security. However, in 1988, Robert Morris compromised `fingerd` by triggering a buffer overflow, allowing him to execute arbitrary code on the target machine. The Morris worm could have been prevented from using `fingerd` as an attack vector by preventing buffer overflows, regardless of whether `fingerd` contained other types of bugs.

By contrast, the function of `/bin/login` is purely related to security. A bug of any kind in `/bin/login` has the potential to allow access where it was not intended. This is security-critical code.

Similarly, in safety-critical code, such as software that runs an X-ray machine, any bug at all could have serious consequences. In practice, then, security-critical and safety-critical code have the same requirements.

There are already standards that address safety-critical code and therefore security-critical code. The problem is that because they must focus on preventing essentially all bugs, they are required to be so strict that most people outside the safety-critical community do not want to use them. This leaves ordinary code like `fingerd` unprotected.

This Technical Specification has two major subdivisions:

— preliminary elements (Clauses 1–4) and

— secure coding rules (Clause 5).

Each secure coding rule in Clause 5 has a separate numbered subsection and a unique section identifier enclosed in brackets (for example, [ptrcomp]). The unique section identifiers are mainly for use by other documents in identifying the rules should the section numbers change because of the addition or elimination of a rule. These identifiers may be used in diagnostics issued by conforming analyzers, but analyzers are not required to do so.

Annexes provide additional information. Annex C (informative) Related Guidelines and References identifies related guidelines and references per rule. A bibliography lists documents referred to during the preparation of this Technical Specification.

The rules documented in this Technical Specification do not rely on source code annotations or assumptions of programmer intent. However, a conforming implementation may take advantage of annotations to inform the analyzer. The rules, as specified, are reasonably simple, although complications can exist in identifying exceptions. An analyzer that conforms to this Technical Specification should be able to analyze code without excessive false positives, even if the code was developed without the expectation that it would be analyzed. Many analyzers provide methods that eliminate the need to research each

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

diagnostic on every invocation of the analyzer. The implementation of such a mechanism is encouraged but not required. This Technical Specification assumes that an analyzer's visibility extends beyond the boundaries of the current function or translation unit being analyzed (see Annex A (informative) Intra- to Interprocedural Transformations).

Completeness and soundness

The rules specified in this Technical Specification are designed to provide a check against a set of programming flaws that are known from practical experience to have led to vulnerabilities. Although rule checking can be performed manually, with increasing program complexity, it rapidly becomes infeasible. For this reason, the use of static analysis tools is recommended.

It should be recognized that, in general, determining conformance to coding rules is computationally undecidable. The precision of static analysis has practical limitations. For example, the *halting theorem* of Computer Science states that there are programs whose exact control flow *cannot* be determined statically. Consequently, any property dependent on control flow—such as halting—may be indeterminate for some programs. A consequence of this undecidability is that it may be impossible for *any* tool to determine statically whether a given rule is satisfied in specific circumstances. The widespread presence of such code may also lead to unexpected results from an analysis tool. Annex D (informative) Decidability of Rules provides information on the decidability of rules in this Technical Specification.

However checking is performed, the analysis may generate

— false negatives: Failure to report a real flaw in the code is usually regarded as the most serious analysis error, as it may leave the user with a false sense of security. Most tools err on the side of caution and consequently generate false positives. However, there may be cases where it is deemed better to report some high-risk flaws and miss others than to overwhelm the user with false positives.

— false positives: The tool reports a flaw when one does not exist. False positives may occur because the code is sufficiently complex that the tool cannot perform a complete analysis. The use of features such as function pointers and libraries may make false positives more likely.

To the greatest extent feasible, an analyzer should be both complete and sound with respect to enforceable rules. An analyzer is considered sound with respect to a specific rule if it cannot give a false-negative result, meaning it finds all violations of a rule within the entire program. An analyzer is considered complete if it cannot issue false-positive results, or false alarms. The possibilities for a given rule are outlined in Table 1.

Table 1 — Completeness and soundness

		False positives	
		Y	N
False negatives	N	Sound with false positives	Complete and sound
	Y	Unsound with false positives	Complete and unsound

The degree to which conforming analyzers minimize false-positive diagnostics is a quality of implementation issue. In other words, quantitative thresholds for false positives and false negatives are outside the scope of this Technical Specification.

Security focus

The purpose of this Technical Specification is to specify analyzable secure coding rules that can be automatically enforced to detect security flaws in C-conforming applications. To be considered a security flaw, a software bug must be triggerable by the actions of a malicious user or attacker. An attacker may trigger a bug by providing malicious data or by providing inputs that execute a particular control

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

path that in turn executes the security flaw. Implementers are encouraged to distinguish violations that operate on untrusted data from those that do not.

Taint analysis

Taint and tainted sources

Certain operations and functions have a domain that is a subset of the type domain of their operands or parameters. When the actual values are outside of the defined domain, the result might be either undefined or at least unexpected. If the value of an operand or argument may be outside the domain of an operation or function that consumes that value, and the value is derived from any external input to the program (such as a command-line argument, data returned from a system call, or data in shared memory), that value is *tainted*, and its origin is known as a *tainted source*. A tainted value is not necessarily known to be out of the domain; rather, it is not known to be in the domain. Only values, and not the operands or arguments, can be tainted; in some cases, the same operand or argument can hold tainted or untainted values along different paths. In this regard, *taint* is an attribute of a value originating from a tainted source.

Restricted sinks

Operands and arguments whose domain is a subset of the domain described by their types are called *restricted sinks*. Any pointer arithmetic operation involving an integer operand is a restricted sink for that operand. Certain parameters of certain library functions are restricted sinks because these functions perform address arithmetic with these parameters, or control the allocation of a resource, or pass these parameters on to another restricted sink. All string input parameters to library functions are restricted sinks because it is possible to pass in a character sequence that is not null terminated. The exceptions are `strncpy` and `strncpy_s`, which explicitly allow the source character sequence not to be null-terminated. For purposes of this Technical Specification, we regard `char *` as a reference to a null-terminated array of characters.

Propagation

Taint is propagated through operations from operands to results unless the operation itself imposes constraints on the value of its result that subsume the constraints imposed by restricted sinks. In addition to operations that propagate the same sort of taint, there are operations that propagate taint of one sort of an operand to taint of a different sort for their results, the most notable example of which is `strlen` propagating the taint of its argument with respect to string length to the taint of its return value with respect to range.

Although the exit condition of a loop is not normally itself considered to be a restricted sink, a loop whose exit condition depends on a tainted value propagates taint to any numeric or pointer variables that are increased or decreased by amounts proportional to the number of iterations of the loop.

Sanitization

To remove the taint from a value, it must be *sanitized* to ensure that it is in the defined domain of any restricted sink into which it flows. Sanitization is performed by *replacement* or *termination*. In replacement, out-of-domain values are replaced by in-domain values, and processing continues using an in-domain value in place of the original. In termination, the program logic terminates the path of execution when an out-of-domain value is detected, often simply by branching around whatever code would have used the value.

In general, sanitization cannot be recognized exactly using static analysis. Analyzers that perform taint analysis usually provide some extralinguistic mechanism to identify sanitizing functions that sanitize an argument (passed by address) in place, return a sanitized version of an argument, or return a status code indicating whether the argument is in the required domain. Because such extralinguistic mechanisms are outside the scope of this specification, this Technical Specification uses a set of rudimentary definitions of sanitization that is likely to recognize real sanitization but might cause nonsanitizing or ineffectively sanitizing code to be misconstrued as sanitizing. The following definition of sanitization presupposes that the analysis is in some way maintaining a set of constraints on each value encountered as the simulated execution progresses: *a given path through the code sanitizes a value with respect to a*

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

given restricted sink if it restricts the range of that value to a subset of the defined domain of the restricted sink type. For example, sanitization of signed integers with respect to an array index operation must restrict the range of that integer value to numbers between zero and the size of the array minus one.

This description is suitable for numeric values, but sanitization of strings with respect to content is more difficult to recognize in a general way.

Tainted source macros

The function-like macros `GET_TAINTED_STRING` and `GET_TAINTED_INTEGER` defined in this section are used in the examples in this Technical Specification to represent one possible method to obtain a tainted string and tainted integer.

```
#define GET_TAINTED_STRING(buf, buf_size)     \
  do {                                         \
    const char *taint = getenv("TAINT");       \
    if (taint == 0) {                          \
      exit(1);                                 \
    }                                          \
                                               \
    size_t taint_size = strlen(taint) + 1;     \
    if (taint_size > buf_size) {               \
      exit(1);                                 \
    }                                          \
                                               \
    strncpy(buf, taint, taint_size);           \
  } while (0)

#define GET_TAINTED_INTEGER(type, val)            \
  do {                                             \
    const char *taint = getenv("TAINT");           \
    if (taint == 0) {                              \
      exit(1);                                     \
    }                                              \
                                                   \
    errno = 0;                                     \
    long tmp = strtol(taint, 0, 10);               \
    if ((tmp == LONG_MIN || tmp == LONG_MAX) &&    \
        errno == ERANGE)                           \
      ; /* retain LONG_MIN or LONG_MAX */          \
    if ((type)-1 < 0) {                            \
      if (tmp < INT_MIN)                           \
        tmp = INT_MIN;                             \
      else if (tmp > INT_MAX)                      \
        tmp = INT_MAX;                             \
    }                                              \
    val = tmp;                                     \
  } while (0)
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Information technology — Programming languages, their environments and system software interfaces — C secure coding rules

1 Scope

This Technical Specification specifies

— rules for secure coding in the C programming language and

— code examples.

This Technical Specification does not specify

— the mechanism by which these rules are enforced or

— any particular coding style to be enforced. (It has been impossible to develop a consensus on appropriate style guidelines. Programmers should define style guidelines and apply these guidelines consistently. The easiest way to consistently apply a coding style is with the use of a code formatting tool. Many interactive development environments provide such capabilities.)

Each rule in this Technical Specification is accompanied by code examples. Code examples are informative only and serve to clarify the requirements outlined in the normative portion of the rule. Examples impose no normative requirements.

Each rule in this Technical Specification that is based on undefined behavior defined in the C Standard identifies the undefined behavior by a numeric code. The numeric codes for undefined behaviors can be found in Annex B, Undefined Behavior.

Two distinct kinds of examples are provided:

— *noncompliant examples* demonstrating language constructs that have weaknesses with potentially exploitable security implications; such examples are expected to elicit a diagnostic from a conforming analyzer for the affected language construct; and

— *compliant examples* are expected not to elicit a diagnostic.

Examples are not intended to be complete programs. For brevity, they typically omit `#include` directives of C Standard Library headers that would otherwise be necessary to provide declarations of referenced symbols. Code examples may also declare symbols without providing their definitions if the definitions are not essential for demonstrating a specific weakness.

Some rules in this Technical Specification have exceptions. Exceptions are part of the specification of these rules and are normative.

2 Conformance

In this Technical Specification, “shall” is to be interpreted as a requirement on an analyzer; conversely, “shall not” is to be interpreted as a prohibition.

Various types of programs (such as compilers or specialized analyzers) can be used to check if a program contains any violations of the coding rules specified in this Technical Specification. In this Technical Specification, all such checking programs are called analyzers. An analyzer can claim conformity with this Technical Specification. Programs that do not yield any diagnostic when analyzed by a conforming analyzer cannot claim conformity to this Technical Specification.

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

A conforming analyzer shall produce a diagnostic for each distinct rule in this Technical Specification upon detecting a violation of that rule, except in the case that the same program text violates multiple rules simultaneously, where a conforming analyzer may aggregate diagnostics but shall produce at least one diagnostic.

NOTE 1 The diagnostic message might be of the form:

```
Accessing freed memory in function abc, file xyz.c, line nnn.
```

NOTE 2 This Technical Specification does not require an analyzer to produce a diagnostic message for any violation of any syntax rule or constraint specified by the C Standard.

Conformance is defined only with respect to source code that is visible to the analyzer. Binary-only libraries, and calls to them, are outside the scope of these rules.

For each rule, the analyzer shall report a diagnostic for at least one program that contains a violation of that rule.

For each rule, the analyzer shall document whether its analysis is guaranteed to report all violations of that rule and shall document its accuracy with respect to avoiding false positives and false negatives.

2.1 Portability assumptions

A conforming analyzer shall be able to diagnose violations of guidelines for at least one C implementation. An analyzer need not diagnose a rule violation if the result is documented for the target implementation and does not cause a security flaw. A conforming analyzer shall document which C implementation is the target.

Variations in quality of implementation permit an analyzer to produce diagnostics concerning portability issues.

EXAMPLE

```
long i;
printf("i = %d", i);
```

This example can produce a diagnostic, such as the mismatch between `%d` and `long int`. This Technical Specification does not specify that a conforming analyzer be complete or sound when diagnosing rule violations. This mismatch might not be a problem for all target implementations, but it is a portability problem because not all implementations have the same representation for `int` and `long`.

3 Normative references

The following documents, in whole or in part, are normatively referenced in this document and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ISO/IEC 9899:2011, *Information technology — Programming languages — C*

ISO 80000-2:2009, *Quantities and units — Part 2: Mathematical signs and symbols to be used in the natural sciences and technology*

ISO/IEC 2382-1:1993, *Information technology — Vocabulary — Part 1: Fundamental terms*

ISO/IEC/IEEE 9945:2009, *Information technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7*

4 Terms and definitions

For the purposes of this document, the terms and definitions given in ISO/IEC 9899:2011, ISO/IEC 2382-1:1993 and the following apply. Other terms are defined where they appear in *italic* type. Mathematical symbols not defined in this document are to be interpreted according to ISO 80000-2:2009.

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

4.1
analyzer
mechanism that diagnoses coding flaws in software programs

Note 1 to entry: Analyzers may include static analysis tools, tools within a compiler suite, or tools in other contexts.

4.2
data flow analysis
tracking of value constraints along nonexcluded paths through the code

Note 1 to entry: Tracking can be performed intraprocedurally, with various assumptions made about what happens at function call boundaries, or interprocedurally, where values are tracked flowing into function calls (directly or indirectly) as arguments and flowing back out either as return values or indirectly through arguments.

Note 2 to entry: Data flow analysis may or may not track values flowing into or out of the heap or take into account global variables. When this specification refers to values flowing, the key point is contrast with variables or expressions, because a given variable or expression may hold different values along different paths, and a given value may be held by multiple variables or expressions along a path.

4.3
exploit
technique that takes advantage of a security vulnerability to violate an explicit or implicit security policy

4.4
in-band error indicator
library function return value on error that can never be returned by a successful call to that library function

4.5
mutilated value
result of an operation performed on an untainted value that yields either an undefined result (such as the result of signed integer overflow), the result of right-shifting a negative number, implicit conversion to an integral type where the value cannot be represented in the destination type, or unsigned integer wrapping

EXAMPLE

```
int j = INT_MAX + 1; // j is mutilated
char c = 1234; // c is mutilated if char is eight bits
unsigned int u = 0U - 1; // u is mutilated
```

Note 1 to entry: A mutilated value can be just as dangerous as a tainted value because it can differ either in sign or magnitude from what the programmer expects.

4.6
nonpersistent signal handler
signal handler running on an implementation that requires the program to again register the signal handler after occurrences of the signal to catch subsequent occurrences of that signal

4.7
out-of-band error indicator
library function return value used to indicate nothing but the error status

4.8
out-of-domain value
one of a set of values that is not in the domain of a particular operator or function

4.9
restricted sink
operands and arguments whose domain is a subset of the domain described by their types

Note 1 to entry: Undefined or unexpected behavior may occur if a tainted value is supplied as a value to a restricted sink.

Note 2 to entry: A diagnostic is required if a tainted value is supplied to a restricted sink.

© ISO/IEC 2013 – All rights reserved

Note 3 to entry: Different restricted sinks may impose different validity constraints for the same value; a given value can be tainted with respect to one restricted sink but sanitized (and consequently no longer tainted) with respect to a different restricted sink.

Note 4 to entry: Specific restricted sinks and requirements for sanitizing tainted values are described in specific rules dealing with taint analysis (see 5.8, 5.14, 5.24, 5.30, 5.39, and 5.46).

4.10
sanitize
assure by testing or replacement that a tainted or other value conforms to the constraints imposed by one or more restricted sinks into which it may flow

Note 1 to entry: If the value does not conform, either the path is diverted to avoid using the value or a different, known-conforming value is substituted.

EXAMPLE Adding a null character to the end of a buffer before passing it as an argument to the `strlen` function.

4.11
security flaw
defect that poses a potential security risk

4.12
security policy
set of rules and practices that specify or regulate how a system or organization provides security services to protect sensitive and critical system resources

4.13
static analysis
any process for assessing code without executing it

Note 1 to entry: See [1], p. 3.

4.14
tainted source
external source of untrusted data

Note 1 to entry: Tainted sources include

— parameters to the `main` function,

— the returned values from `localeconv`, `fgetc`, `getc`, `getchar`, `fgetwc`, `getwc`, and `getwchar`, and

— the strings produced by `getenv`, `fscanf`, `vfscanf`, `vscanf`, `fgets`, `fread`, `fwscanf`, `vfwscanf`, `vwscanf`, `wscanf`, and `fgetws`.

4.15
tainted value
value derived from a tainted source that has not been sanitized

4.16
target implementation
implementation of the C programming language whose environmental limits and implementation-defined behavior are assumed by the analyzer during the analysis of a program

4.17
UB
undefined behavior

4.18
unexpected behavior
well-defined behavior that may be unexpected or unanticipated by the programmer; incorrect programming assumptions

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

4.19
unsigned integer wrapping
computation involving unsigned operands whose result is reduced modulo the number that is one greater than the largest value that can be represented by the resulting type

4.20
untrusted data
data originating from outside of a trust boundary

Note 1 to entry: See [2].

4.21
valid pointer
pointer that refers to an element within an array or one past the last element of an array

Note 1 to entry: For the purposes of this definition, a pointer to an object that is not an element of an array behaves the same as a pointer to the first element of an array of length one with the type of the object as its element type. (See ISO/IEC 9899:2011, 6.5.8, paragraph 4.)

Note 2 to entry: For the purposes of this definition, an object can be considered to be an array of a certain number of bytes; that number is the size of the object, as produced by the `sizeof` operator. (See ISO/IEC 9899:2011, 6.3.2.3, paragraph 7.)

4.22
vulnerability
set of conditions that allows an attacker to violate an explicit or implicit security policy

5 Rules

5.1 Accessing an object through a pointer to an incompatible type [ptrcomp]

Rule

Accessing an object through a pointer to an incompatible type (other than `unsigned char`) shall be diagnosed.

Rationale

ISO/IEC 9899:2011, 6.5, paragraph 7, states,

> *An object shall have its stored value accessed only by an lvalue expression that has one of the following types:*
>
> — *a type compatible with the effective type of the object,*
>
> — *a qualified version of a type compatible with the effective type of the object,*
>
> — *a type that is the signed or unsigned type corresponding to the effective type of the object,*
>
> — *a type that is the signed or unsigned type corresponding to a qualified version of the effective type of the object,*
>
> — *an aggregate or union type that includes one of the aforementioned types among its members (including, recursively, a member of a subaggregate or contained union), or*
>
> — *a character type.*

The intent of this list is to specify those circumstances in which an object may or may not be aliased.

According to ISO/IEC 9899:2011, 6.2.6.1,

© ISO/IEC 2013 – All rights reserved

Certain object representations need not represent a value of the object type. If the stored value of an object has such a representation and is read by an lvalue expression that does not have character type, the behavior is undefined.

Accessing an object through a pointer to an incompatible type (other than `unsigned char`) is undefined behavior.

C identifies the following undefined behavior:

UB	Description
37	An object has its stored value accessed other than by an lvalue of an allowable type (6.5).

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because an object of type `float` is incremented through a pointer to `int`, `ip`.

```
void f(void) {
  if (sizeof(int) == sizeof(float)) {
    float f = 0.0f;
    int *ip = (int *)&f;

    printf("float is %f\n", f);

    (*ip)++;  // diagnostic required

    printf("float is %f\n", f);
  }
}
```

5.2 Accessing freed memory [accfree]

Rule

After an allocated block of dynamic storage has been deallocated by a memory management function, the evaluation of any pointers into the freed memory, including being dereferenced or acting as an operand of an arithmetic operation, type cast, or right-hand side of an assignment, shall be diagnosed.

Rationale

C identifies the situation in which undefined behavior arises as a result of accessing freed memory:

UB	Description
177	The value of a pointer that refers to space deallocated by a call to the `free` or `realloc` function is used (7.22.3).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because `head->next` is accessed after `head` has been freed.

```
struct List { struct List *next; /* ... */ };

void free_list(struct List *head) {
  for (; head != NULL; head = head->next) {  // diagnostic required
    free(head);
  }
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because `buf` is written to after it has been freed.

```
int main(int argc, char *argv[]) {
  if (argc < 2) {
```

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
    /* ... */
  }

  char *return_val = 0;

  const size_t bufsize = strlen(argv[1]) + 1;

  char *buf = (char *)malloc(bufsize);
  if (!buf) {
    /* ... */
  }
  /* ... */
  free(buf);
  /* ... */
  return_val = strncpy(buf, argv[1], bufsize);  // diagnostic required
  if (return_val) {
    /* ... */
  }
  return EXIT_SUCCESS;
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because `realloc` may free `c_str1` when it returns `NULL`, resulting in `c_str1` being freed twice.

```
void f(char * c_str1, size_t size) {
  char * c_str2 = (char *)realloc(c_str1, size);
  if (c_str2 == NULL) {
    free(c_str1); // diagnostic required
    return;
  }
}
```

5.3 Accessing shared objects in signal handlers [accsig]

Rule

Accessing values of objects that are neither lock-free atomic objects nor of type `volatile sig_atomic_t` in a signal handler shall be diagnosed.

Rationale

C identifies the situation in which undefined behavior arises as a result of accessing a static storage duration object without the correct characteristics:

UB	Description
132	A signal occurs other than as the result of calling the `abort` or `raise` function, and the signal handler refers to an object with static storage duration other than by assigning a value to an object declared as `volatile sig_atomic_t`, or calls any function in the standard library other than the `abort` function, the `_Exit` function, or the `signal` function (for the same signal number) (7.14.1.1).

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the object referred to by the shared pointer `err_msg` is accessed from the signal handler `handler` via the C Standard Library function `strcpy`.

```
#define MAX_MSG_SIZE 24
char *err_msg;

void handler(int signum) {
  if ((strcpy(err_msg, "SIGINT detected.")) == err_msg){ // diagnostic required
    /* ... */
  }
}

int main(void) {

  signal(SIGINT, handler);
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
  err_msg = (char *)malloc(MAX_MSG_SIZE);
  if (err_msg == NULL) {
    /* Handle error condition */
  }
  if ((strcpy(err_msg, "No errors yet.")) == err_msg) {
    /* ... */
  }

  /* Main code loop */

  return EXIT_SUCCESS;
}
```

5.4 No assignment in conditional expressions [boolasgn]

Rule

The use of the assignment operator in the following contexts shall be diagnosed:

— `if` (controlling expression)

— `while` (controlling expression)

— `do ... while` (controlling expression)

— `for` (second operand)

— `?:` (first operand)

— `&&` (either operand)

— `||` (either operand)

— comma operator (second operand) when the comma expression is used in any of these contexts

— `?:` (second or third operands) where the ternary expression is used in any of these contexts

Rationale

Mistyping or erroneously using = in Boolean expressions, where == was intended, is a common cause of program error. This rule makes the presumption that any use of = was intended to be == unless the context makes it clear that such is not the case.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the expression `x = y` is used as the controlling expression of the `while` statement.

```
while ( x = y ) { /* ... */ }  //  diagnostic required
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the expression `x = y` is used as the controlling expression of the `while` statement.

```
do { /* ... */ } while ( foo(), x = y ) ; // diagnostic required
```

EXAMPLE 3 In this compliant example, no diagnostic is required because the expression `x = y` is not used as the controlling expression of the `while` statement.

```
do { /* ... */ } while ( x = y, p == q ) ; // no diagnostic required
```

Exceptions

— EX1: Assignment is permitted where the result of the assignment is itself a parameter to a comparison expression (e.g., `x == y` or `x != y`) or relational expression and need not be diagnosed.

EXAMPLE This example shows an acceptable use of this exception.

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.


```
if ( ( x = y )  != 0  ) { /* ... */ }
```

— EX2: Assignment is permitted where the expression consists of a single primary expression.

EXAMPLE 1 This example shows an acceptable use of this exception.

```
if ( ( x = y ) ) { /* ... */ }
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because `&&` is not a comparison or relational operator and the entire expression is not primary.

```
if ( ( v = w ) && flag ) { /* ... */ }  // diagnostic required
```

— EX3: Assignment is permitted in the above contexts where it occurs in a function argument or array index.

EXAMPLE This example shows an acceptable use of this exception.

```
if ( foo( x = y ) ) { /* ... */ }
```

5.5 Calling functions in the C Standard Library other than `abort`, `_Exit`, and `signal` from within a signal handler [asyncsig]

Rule

Calling functions in the C Standard Library other than `abort`, `_Exit`, and `signal` from within a signal handler shall be diagnosed.

Rationale

C identifies the situation in which undefined behavior arises as a result of calling other C library functions:

UB	Description
131	A signal occurs as the result of calling the `abort` or `raise` function, and the signal handler calls the `raise` function (7.14.1.1).
132	A signal occurs other than as the result of calling the `abort` or `raise` function, and the signal handler refers to an object with static storage duration other than by assigning a value to an object declared as `volatile sig_atomic_t`, or calls any function in the standard library other than the `abort` function, the `_Exit` function, or the `signal` function (for the same signal number) (7.14.1.1).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the C Standard Library function `fprintf` is called from the signal handler `handler` via the function `log_message`.

```
#define MAXLINE 1024

char info[MAXLINE];

void log_message(void) {
  fprintf(stderr, "%s\n", info);  // diagnostic required
}

void handler(int signum) {
  log_message();
}

int main(void) {
  if (signal(SIGINT, handler) == SIG_ERR) {
    /* Handle error */
  }
  /* An interactive attention signal might invoke handler() from here on. */
  while (1) {
    /* Main loop program code */
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
    log_message();

    /* More program code */
  }
  return EXIT_SUCCESS;
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the C Standard Library function `raise` is called from the signal handler `int_handler`.

```
void term_handler(int signum) {
  /* SIGTERM handling specific */
}

void int_handler(int signum) {
  /* SIGINT handling specific */
  if (raise(SIGTERM) != 0) {  // diagnostic required
    /* Handle error */
  }
}

int main(void) {
  if (signal(SIGTERM, term_handler) == SIG_ERR) {
    /* Handle error */
  }
  if (signal(SIGINT, int_handler) == SIG_ERR) {
    /* Handle error */
  }

  /* Program code */
  if (raise(SIGINT) != 0) {
    /* Handle error */
  }
  /* More code */

  return EXIT_SUCCESS;
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the C Standard Library function `longjmp` is called from the signal handler `handler`.

```
#define MAXLINE 1024

static jmp_buf env;

void handler(int signum) {
  longjmp(env, 1);  // diagnostic required
}

void log_message(char *info1, char *info2) {
  static char *buf = NULL;
  static size_t bufsize;
  char buf0[MAXLINE];

  if (buf == NULL) {
    buf = buf0;
    bufsize = sizeof(buf0);
  }

  /*
   *  Try to fit a message into buf, else re-allocate
   *  it on the heap and then log the message.
   */

/*** VULNERABILITY IF SIGINT RAISED HERE ***/

  if (buf == buf0) {
    buf = NULL;
  }
}
```

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.


```
int main(void) {
  if (signal(SIGINT, handler) == SIG_ERR) {
    /* Handle error */
  }

  char *info1;
  char *info2;

  /* info1 and info2 are set by user input here */

  if (setjmp(env) == 0) {
    while (1) {
      /* Main loop program code */
      log_message(info1, info2);
      /* More program code */
    }
  }
  else {
    log_message(info1, info2);
  }

  return EXIT_SUCCESS;
}
```

5.6 Calling functions with incorrect arguments [argcomp]

Rule

Calling a function with the wrong number or type of arguments shall be diagnosed.

Rationale

C identifies four distinct situations in which undefined behavior may arise as a result of invoking a function using a declaration that is incompatible with its definition or with incorrect types or numbers of arguments:

UB	Description
26	A pointer is used to call a function whose type is not compatible with the pointed-to type (6.3.2.3).
38	For a call to a function without a function prototype in scope, the number of arguments does not equal the number of parameters (6.5.2.2).
39	For a call to a function without a function prototype in scope where the function is defined with a function prototype, either the prototype ends with an ellipsis or the types of the arguments after promotion are not compatible with the types of the parameters (6.5.2.2).
41	A function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called function (6.5.2.2).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the C Standard Library function `strchr` is called through the function pointer `fp` with incorrectly typed arguments.

```
char *(*fp)();

void f(void) {
  char *c;
  fp = strchr;
  c = fp(12, 2);  // diagnostic required
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the function `copy` is defined to take two arguments but is called with three arguments.

```
/* in another source file */
void copy(char *dst, const char *src) {
  if (!strcpy(dst, src)) {
```

© ISO/IEC 2013 – All rights reserved

```
    /* report error */
  }
}
```

```
/* in this source file -- no copy prototype in scope */
void copy();

void g(const char *s) {
  char buf[20];
  copy(buf, s, sizeof buf);  // diagnostic required
  /* ... */
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the function `buginf` is defined to take a variable number of arguments but is declared in another file with no prototype and is called.

```
/* in another source file */
void buginf(const char *fmt, ...) {
   /* ... */
}
```

```
/* in this source file -- no buginf prototype in scope */
void buginf();

void h(void) {
  buginf("bug in function %s, line %d\n", __func__, __LINE__);  // diagnostic required
  /* ... */
}
```

EXAMPLE 4 In this noncompliant example, a diagnostic is required because the function `f` is defined to take an argument of type `long`, but `f` is called from another file with an argument of type `int`.

```
/* in somefile.c */

long f(long x) {
  return x < 0 ? -x : x;
}
```

```
/* in otherfile.c */

int g(int x) {
  return f(x);  // diagnostic required
}
```

5.7 Calling `signal` from interruptible signal handlers [sigcall]

Rule

On systems with nonpersistent signal handlers, calling `signal` from within a signal handler whose execution can be interrupted by receipt of a signal shall be diagnosed.

Rationale

Calling `signal` under these conditions presents a race condition.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required on implementations with nonpersistent signal handlers because the C Standard Library function `signal` is called from the signal handler `handler`.

```
void handler(int signum) {
  if (signal(signum, handler) == SIG_ERR) {  // diagnostic required
```

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
    /* ... */
  }

  /* ... */
}

void f(void) {
  if (signal(SIGUSR1, handler) == SIG_ERR) {
    /* ... */
  }

  /* ... */
}
```

5.8 Calling `system` [syscall]

Rule

All calls to the `system` function shall be diagnosed.

Rationale

Use of the `system` function can result in exploitable vulnerabilities

— when passing an unsanitized or improperly sanitized command string originating from a tainted source, or

— if a command is specified without a path name and the command processor path name resolution mechanism is accessible to an attacker, or

— if a relative path to an executable is specified and control over the current working directory is accessible to an attacker, or

— if the specified executable program can be spoofed by an attacker.

Although exceptions to this rule are necessary, they can only be identified on a case-by-case basis during a code review and are consequently outside the scope of this rule.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because a string consisting of `any_cmd` and the tainted value stored in `input` is copied into `cmdbuf` and then passed as an argument to the `system` function to execute.

```
void f(char *input) {
  char cmdbuf[512];
  int len_wanted = snprintf(
    cmdbuf, sizeof(cmdbuf), "any_cmd '%s'", input
  );

  if (len_wanted >= sizeof(cmdbuf)) {
    perror("Input too long");
  } else if (len_wanted < 0) {
    perror("Encoding error");
  } else if (system(cmdbuf) == -1) {  // diagnostic required
    perror("Error executing input");
  }
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because `system` is used to remove the `.config` file in the user's home directory.

```
void g(void) {
  system("rm ~/.config");  // diagnostic required
}
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

5.9 Comparison of padding data [padcomp]

Rule

Comparison of padding data shall be diagnosed.

Rationale

The value of padding bits is unspecified and may contain data initially provided by an attacker.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the C Standard Library function `memcmp` is used to compare the structures `s1` and `s2`, including padding data.

```
struct my_buf {
  char buff_type;
  size_t size;
  char buffer[50];
};

unsigned int buf_compare(
  const struct my_buf *s1,
  const struct my_buf *s2)
{
  if (!memcmp(s1, s2, sizeof(struct my_buf))) {  // diagnostic required
    /* ... */
  }

  return 0;
}
```

5.10 Converting a pointer to integer or integer to pointer [intptrconv]

Rule

Converting an integer type to a pointer type shall be diagnosed if the resulting pointer is incorrectly aligned, does not point to an entity of the referenced type, or is a trap representation.

Converting a pointer type to an integer type shall be diagnosed if the result cannot be represented in the integer type.

Rationale

C identifies the situation in which undefined behavior arises as a result of converting a pointer to an integer type:

UB	Description
24	Conversion of a pointer to an integer type produces a value outside the range that can be represented (6.3.2.3).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required on an implementation where pointers are 64 bits and unsigned integers are 32 bits because the result of converting the 64-bit `ptr` cannot be represented in the 32-bit integer type.

```
void f(void) {
  char *ptr;
  /* ... */
  unsigned int number = (unsigned int)ptr;  // diagnostic required
  /* ... */
}
```

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the conversion of the integer literal `0xdeadbeef` to a pointer that is not known to point to an entity of the referenced type.

```
unsigned int *g(void) {
  unsigned int *ptr = (unsigned int *)0xdeadbeef;  // diagnostic required
  /* ... */
  return ptr;
}
```

Exceptions

— EX1: A null pointer can be converted to an integer; it takes on the value 0. Likewise, a 0 integer can be converted to a pointer; it becomes the null pointer.

— EX2: Any valid pointer (4.21) to `void` can be converted to `intptr_t` or `uintptr_t` and back with no change in value. (This includes the underlying types if `intptr_t` and `uintptr_t` are `typedefs` and any `typedefs` that denote the same types as `intptr_t` and `uintptr_t`.)

EXAMPLE

```
void h(void) {
  intptr_t i = (intptr_t)(void *)&i;
  uintptr_t j = (uintptr_t)(void *)&j;

  void *ip = (void *)i;
  void *jp = (void *)j;

  assert(ip == &i);
  assert(jp == &j);
}
```

5.11 Converting pointer values to more strictly aligned pointer types [alignconv]

Rule

Converting a pointer value to a pointer type that is more strictly aligned than the type the value actually points to shall be diagnosed.

Rationale

Converting a pointer value to a pointer type that is more strictly aligned than the type the value actually points to results in undefined behavior if the actual value is unaligned with respect to the destination type.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the `char` pointer `&c` is converted to the more strictly aligned int pointer `i_ptr`.

```
void f(void) {
  int *i_ptr;
  char c;

  i_ptr = (int *)&c;  // diagnostic required
  /* ... */
}
```

EXAMPLE 2 In this compliant example, a diagnostic is not required because the value referenced by the `char` pointer `c_ptr` has the alignment of type `int`.

```
void f(void) {
  char *c_ptr;
  int *i_ptr;
  int i;

  c_ptr = (char *)&i;
  i_ptr = (int *)c_ptr;
  /* ... */
```

© ISO/IEC 2013 – All rights reserved

```
}
```

5.12 Copying a FILE object [filecpy]

Rule

Copying a FILE object shall be diagnosed.

Rationale

According to ISO/IEC 9899:2011, 7.21.3, paragraph 6,

> *The address of the FILE object used to control a stream may be significant; a copy of a FILE object need not serve in place of the original.*

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the FILE object stdout is copied.

```
int main(void) {
  FILE my_stdout = *(stdout);  // diagnostic required
  if (fputs("Hello, World!\n", &my_stdout) == EOF) {
    /* ... */
  }
  return EXIT_SUCCESS;
}
```

5.13 Declaring the same function or object in incompatible ways [funcdecl]

Rule

Two or more incompatible declarations of the same function or object that appear in the same program shall be diagnosed.

Rationale

C identifies three distinct situations in which undefined behavior may arise as a result of incompatible declarations of the same function or object:

UB	Description
15	Two declarations of the same object or function specify types that are not compatible (6.2.7).
37	An object has its stored value accessed other than by an lvalue of an allowable type (6.5).
41	A function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called function (6.5.2.2).

While the effects of two incompatible declarations simply appearing in the same program may be benign on most implementations, the effects of invoking a function through an expression whose type is incompatible with the function definition are typically catastrophic. Similarly, the effects of accessing an object using an lvalue of a type that is incompatible with the object definition may range from unintended information exposure to memory overwrite to a hardware trap.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the variable i has two incompatible declarations.

```
/* in a.c */
extern int i;  // diagnostic required

int f(void) {
  return ++i;
}
```

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.


```
/* in b.c */
short i;  // diagnostic required
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the variable `a` has two incompatible declarations.

```
/* in a.c */
extern int *a;  // diagnostic required

int g(unsigned i, int x) {
  int tmp = a[i];
  a[i] = x;
  return tmp;
}

/* in b.c */
int a[] = { 1, 2, 3, 4 };  // diagnostic required
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the function `h` has two incompatible declarations.

```
/* in a.c */
extern int h(int a);  // diagnostic required

int main(void) {
  printf("%d", h(10));
  return EXIT_SUCCESS;
}

/* in b.c */
long h(long a) {  // diagnostic required
  return a * 2;
}
```

EXAMPLE 4 According to ISO/IEC 9899:2011, 5.2.4.1, external identifiers need to be unique only within the first 31 characters. In this noncompliant example, a diagnostic is required on implementations where the external identifiers `bash_groupname_completion_function` and `bash_groupname_completion_funct` are identical, because it results in incompatible declarations.

```
/* in bash/bashline.h */
extern char* bash_groupname_completion_function(const char *, int);
// diagnostic required
// the identifier exceeds 31 characters
/* in a.c */
#include <bashline.h>

void w(const char *s, int i) {
  bash_groupname_completion_function(s, i);
}

/* in b.c */
int bash_groupname_completion_funct;  // diagnostic required
// identifier not unique within 31 characters
```

NOTE The identifier `bash_groupname_completion_function` referenced here was taken from GNU Bash version 3.2.

Exception

No diagnostic need be issued if a declaration that is incompatible with the definition occurs in a translation unit that does not contain any definition or uses of the function or object other than additional declarations, if any.

EXAMPLE

```
/* a.c: */
int x = 0; /* the definition */

/* b.c: */
```

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
extern char x; /* incompatible declaration */
/* but no other references to 'x' */
```

5.14 Dereferencing an out-of-domain pointer [nullref]

Rule

Dereferencing a tainted or out-of-domain pointer shall be diagnosed.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because if `malloc` returns `NULL`, then the call to `memcpy` will dereference the null pointer `c_str`.

```
void f(const char *input_str) {
  size_t size = strlen(input_str) + 1;
  char *c_str = (char *)malloc(size);
  if ((memcpy(c_str, input_str, size)) == c_str) {  // diagnostic required
    /* ... */
  }
  /* ... */
  free(c_str);
  c_str = NULL;
}
```

5.15 Escaping of the address of an automatic object [addrescape]

Rule

The address of an object with automatic storage duration returned from a function or held in any pointer variable whose lifetime extends past the lifetime of the referenced object at the time the automatic object goes out of scope shall be diagnosed.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the address of the automatic object `c_str` remains in the pointer variable `p` when `c_str` goes out of scope in the function `dont_do_this`.

```
const char *p;
void dont_do_this(void) {
  const char c_str[] = "This will change";
  p = c_str;  // diagnostic required
}

void innocuous(void) {
  const char c_str[] = "Surprise, surprise";
  puts(c_str);
}

int main(void) {
  dont_do_this();
  innocuous();
  puts(p);

  return EXIT_SUCCESS;
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the address of the automatic object `array` is returned.

```
int *init_array(void) {
  int array[10] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10 };
  return array;  // diagnostic required
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the address of the automatic object `fmt` remains in the pointer variable `ptr_param` when `fmt` goes out of scope in the function `squirrel_away`.

 © ISO/IEC 2013 – All rights reserved

```
void squirrel_away(char **ptr_param) {
  char fmt[] = "Error: %s\n";

  /* ... */
  *ptr_param = fmt;  // diagnostic required
}

int main(void) {
  char *ptr;
  squirrel_away(&ptr);

  /* ... */
  return EXIT_SUCCESS;
}
```

5.16 Conversion of signed characters to wider integer types before a check for EOF [signconv]

Rule

Converting a tainted value of type `char` or `signed char` to a larger integer type without having first cast the value to `unsigned char` shall be diagnosed if the value is subsequently compared with the value of `EOF`.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the character of type `char` pointed to by `c_str` is converted to `int` without first being cast to `unsigned char`.

```
int yy_string_get(char *c_str) {
  int c = EOF;

  if (c_str && *c_str) {
    c = *c_str++;  // if char is signed, a 0xFF char can be confused with EOF
  }
  return c;
}

/* ... */

char string[BUFSIZ];
GET_TAINTED_STRING(string, BUFSIZ);
if (yy_string_get(string) == EOF) // diagnostic required
```

5.17 Use of an implied default in a switch statement [swtchdflt]

Rule

A `switch` statement with a controlling expression of enumerated type that does not include a default case and does not include cases for all enumeration constants of that type shall be diagnosed.

Rationale

A `switch` statement with a controlling expression of enumerated type that does not include a default case and does not include cases for all enumeration constants of that type indicates logical incompleteness.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because not all possible values of `widget_type` are checked for in the `switch` statement.

```
enum WidgetEnum { WE_W, WE_X, WE_Y, WE_Z };

void f(enum WidgetEnum widget_type) {
  switch (widget_type) {  // diagnostic required
    case WE_X:
      /* ... */
```


Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
      break;
    case WE_Y:
      /* ... */
      break;
    case WE_Z:
      /* ... */
      break;
  }
}
```

5.18 Failing to close files or free dynamic memory when they are no longer needed [fileclose]

Rule

A call to the `fopen` or `freopen` function shall be diagnosed after the lifetime of the last pointer object that stores the return value of the call has ended without a call to `fclose` with that pointer value.

A call to a standard memory allocation function shall be diagnosed after the lifetime of the last pointer object that stores the return value of the call has ended without a call to a standard memory deallocation function with that pointer value.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the resource allocated by the call to `fopen` is not closed.

```
int f(void) {
  const char *filename = "secure.dat";

  FILE *f = fopen(filename, "r");  // diagnostic required
  if (f == NULL) {
    /* ... */
  }

  /* ... */
  return 0;
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the resource allocated by the call to `malloc` is not freed.

```
int f(void) {
  char *text_buffer = (char *)malloc(BUFSIZ); // diagnostic required

  if (text_buffer == NULL) {
    return -1;
  }
  return 0;
}
```

5.19 Failing to detect and handle standard library errors [liberr]

Rule

Failure to branch conditionally on detection or absence of a standard library error condition shall be diagnosed.

The successful completion or failure of each of the standard library functions listed in Table 2 shall be determined either by comparing the function's return value with the value listed in the column labeled "Error Return" or by calling one of the library functions mentioned in the footnotes to the same column.

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Table 2 — Library functions and returns

Function	Successful return	Error return
aligned_alloc	pointer to space	NULL
asctime_s	zero	nonzero
at_quick_exit	zero	nonzero
atexit	zero	nonzero
bsearch	pointer to matching element	NULL
bsearch_s	pointer to matching element	NULL
btowc	converted wide character	WEOF
c16rtomb	number of bytes	(size_t)(-1)
c32rtomb	number of bytes	(size_t)(-1)
calloc	pointer to space	NULL
clock	processor time	(clock_t)(-1)
cnd_broadcast	thrd_success	thrd_error
cnd_init	thrd_success	thrd_nomem or thrd_error
cnd_signal	thrd_success	thrd_error
cnd_timedwait	thrd_success	thrd_timedout or thrd_error
cnd_wait	thrd_success	thrd_error
ctime_s	zero	nonzero
fclose	zero	EOF (negative)
fflush	zero	EOF (negative)
fgetc	character read	EOF[a]
fgetpos	zero	nonzero
fgets	pointer to string	NULL
fgetwc	wide character read	WEOF[a]
fopen	pointer to stream	NULL
fopen_s	zero	nonzero
fprintf	number of characters (nonnegative)	negative
fprintf_s	number of characters (nonnegative)	negative
fputc	character written	EOF[b]
fputs	nonnegative	EOF (negative)
fputws	nonnegative	EOF (negative)
fread	elements read	elements read
freopen	pointer to stream	NULL
freopen_s	zero	nonzero
fscanf	number of conversions (nonnegative)	EOF (negative)
fscanf_s	number of conversions (nonnegative)	EOF (negative)
fseek	zero	nonzero
fsetpos	zero	nonzero
ftell	file position	–1L
fwprintf	number of wide characters (nonnegative)	negative
fwprintf_s	number of wide characters (nonnegative)	negative
fwrite	elements written	elements written

© ISO/IEC 2013 – All rights reserved

Table 2 *(continued)*

Function	Successful return	Error return
fwscanf	number of conversions (nonnegative)	EOF (negative)
fwscanf_s	number of conversions (nonnegative)	EOF (negative)
getc	character read	EOF[a]
getchar	character read	EOF[a]
getenv	pointer to string	NULL
getenv_s	pointer to string	NULL
gets_s	pointer to string	NULL
getwc	wide character read	WEOF
getwchar	wide character read	WEOF
gmtime	pointer to broken-down time	NULL
gmtime_s	pointer to broken-down time	NULL
localtime	pointer to broken-down time	NULL
localtime_s	pointer to broken-down time	NULL
malloc	pointer to space	NULL
mblen, s != NULL	number of bytes	–1
mbrlen, s != NULL	number of bytes or status	(size_t)(-1)
mbrtoc16	number of bytes or status	(size_t)(-1),errno == EILSEQ
mbrtoc32	number of bytes or status	(size_t)(-1),errno == EILSEQ
mbrtowc, s != NULL	number of bytes or status	(size_t)(-1),errno == EILSEQ
mbsrtowcs	number of non-null elements	(size_t)(-1),errno == EILSEQ
mbsrtowcs_s	zero	nonzero
mbstowcs	number of non-null elements	(size_t)(-1)
mbstowcs_s	zero	nonzero
mbtowc, s != NULL	number of bytes	–1
memchr	pointer to located character	NULL
mktime	calendar time	(time_t)(-1)
mtx_init	thrd_success	thrd_error
mtx_lock	thrd_success	thrd_error
mtx_timedlock	thrd_success	thrd_timedout or thrd_error
mtx_trylock	thrd_success	thrd_busy or thrd_error
mtx_unlock	thrd_success	thrd_error
printf_s	number of characters (nonnegative)	negative
putc	character written	EOF[b]
putwc	wide character written	WEOF
raise	zero	nonzero
realloc	pointer to space	NULL
remove	zero	nonzero
rename	zero	nonzero
setlocale	pointer to string	NULL
setvbuf	zero	nonzero
scanf	number of conversions (nonnegative)	EOF (negative)

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Table 2 *(continued)*

Function	Successful return	Error return
scanf_s	number of conversions (nonnegative)	EOF (negative)
signal	pointer to previous function	SIG_ERR, errno > 0
snprintf	number of characters that would be written (nonnegative)	negative
snprintf_s	number of characters that would be written (nonnegative)	negative
sprintf	number of non-null characters written	negative
sprintf_s	number of non-null characters written	negative
sscanf	number of conversions (nonnegative)	EOF (negative)
sscanf_s	number of conversions (nonnegative)	EOF (negative)
strchr	pointer to located character	NULL
strerror_s	zero	nonzero
strftime	number of non-null characters	zero
strpbrk	pointer to located character	NULL
strrchr	pointer to located character	NULL
strstr	pointer to located string	NULL
strtod	converted value	zero, errno == ERANGE
strtof	converted value	zero, errno == ERANGE
strtoimax	converted value	INTMAX_MAX or INTMAX_MIN, errno == ERANGE
strtok	pointer to first character of a token	NULL
strtok_s	pointer to first character of a token	NULL
strtol	converted value	LONG_MAX or LONG_MIN, errno == ERANGE
strtold	converted value	zero, errno == ERANGE
strtoll	converted value	LLONG_MAX or LLONG_MIN, errno == ERANGE
strtoumax	converted value	UINTMAX_MAX, errno == ERANGE
strtoul	converted value	ULONG_MAX, errno == ERANGE
strtoull	converted value	ULLONG_MAX, errno == ERANGE
strxfrm	length of transformed string	>= n
swprintf	number of non-null wide characters	negative
swprintf_s	number of non-null wide characters	negative
swscanf	number of conversions (nonnegative)	EOF (negative)
swscanf_s	number of conversions (nonnegative)	EOF (negative)
thrd_create	thrd_success	thrd_nomem or thrd_error
thrd_detach	thrd_success	thrd_error
thrd_join	thrd_success	thrd_error
thrd_sleep	zero	negative
time	calendar time	(time_t)(-1)
timespec_get	base	zero
tmpfile	pointer to stream	NULL

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Table 2 *(continued)*

Function	Successful return	Error return
tmpfile_s	zero	nonzero
tmpnam	non-null pointer	NULL
tmpnam_s	zero	nonzero
tss_create	thrd_success	thrd_error
tss_get	value of thread-specific storage	zero
tss_set	thrd_success	thrd_error
ungetc	character pushed back	EOF (negative; see below)
ungetwc	character pushed back	WEOF (negative)
vfprintf	number of characters (nonnegative)	negative
vfprintf_s	number of characters (nonnegative)	negative
vfscanf	number of conversions (nonnegative)	EOF (negative)
vfscanf_s	number of conversions (nonnegative)	EOF (negative)
vfwprintf	number of wide characters (nonnegative)	negative
vfwprintf_s	number of wide characters (nonnegative)	negative
vfwscanf	number of conversions (nonnegative)	EOF (negative)
vfwscanf_s	number of conversions (nonnegative)	EOF (negative)
vprintf_s	number of characters (nonnegative)	negative
vscanf	number of conversions (nonnegative)	EOF (negative)
vscanf_s	number of conversions (nonnegative)	EOF (negative)
vsnprintf	number of characters that would be written (nonnegative)	negative
vsnprintf_s	number of characters that would be written (nonnegative)	negative
vsprintf	number of non-null characters (nonnegative)	negative
vsprintf_s	number of non-null characters (nonnegative)	negative
vsscanf	number of conversions (nonnegative)	EOF (negative)
vsscanf_s	number of conversions (nonnegative)	EOF (negative)
vswprintf	number of non-null wide characters	negative
vswprintf_s	number of non-null wide characters	negative
vswscanf	number of conversions (nonnegative)	EOF (negative)
vswscanf_s	number of conversions (nonnegative)	EOF (negative)
vwprintf_s	number of wide characters (nonnegative)	negative
vwscanf	number of conversions (nonnegative)	EOF (negative)
vwscanf_s	number of conversions (nonnegative)	EOF (negative)
wcrtomb	number of bytes stored	(size_t)(-1)
wcschr	pointer to located wide character	NULL
wcsftime	number of non-null wide characters	zero
wcspbrk	pointer to located wide character	NULL
wcsrchr	pointer to located wide character	NULL
wcsrtombs	number of non-null bytes	(size_t)(-1),errno == EILSEQ

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Table 2 *(continued)*

Function	Successful return	Error return
wcsrtombs_s	zero	nonzero
wcsstr	pointer to located wide string	NULL
wcstod	converted value	zero, errno == ERANGE
wcstof	converted value	zero, errno == ERANGE
wcstoimax	converted value	INTMAX_MAX or INTMAX_MIN, errno == ERANGE
wcstok	pointer to first wide character of a token	NULL
wcstok_s	pointer to first wide character of a token	NULL
wcstol	converted value	LONG_MAX or LONG_MIN, errno == ERANGE
wcstold	converted value	zero, errno == ERANGE
wcstoll	converted value	LLONG_MAX or LLONG_MIN, errno == ERANGE
wcstombs	number of non-null bytes	(size_t)(-1)
wcstombs_s	zero	nonzero
wcstoumax	converted value	UINTMAX_MAX, errno == ERANGE
wcstoul	converted value	ULONG_MAX, errno == ERANGE
wcstoull	converted value	ULLONG_MAX, errno == ERANGE
wcsxfrm	length of transformed wide string	>= n
wctob	converted character	EOF
wctomb, s != NULL	number of bytes stored	–1
wctomb_s, s != NULL	number of bytes stored	–1
wctrans	valid argument to towctrans	zero
wctype	valid argument to iswctype	zero
wmemchr	pointer to located wide character	NULL
wprintf_s	number of wide characters (nonnegative)	negative
wscanf	number of conversions (nonnegative)	EOF (negative)
wscanf_s	number of conversions (nonnegative)	EOF (negative)

[a] Use feof and ferror.
[b] Use ferror.

The ungetc function does not set the error indicator, even when it fails, so it is not possible to check for errors reliably unless it is known that the argument is not equal to EOF. C states that "one character of pushback is guaranteed," so this should not be an issue if, at most, one character is ever pushed back before reading again.

NOTE A cumulative error check satisfies the rule as long as undefined behavior is not triggered (for example, by using the contents of the fgets or fgetws array or using the file position indicator after fread or fwrite without first checking for error).

Rationale

Failure to branch conditionally on detection or absence of a standard library error condition can result in unexpected behavior.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the return value of fseek is not checked for an error condition.

© ISO/IEC 2013 – All rights reserved

```
void test_unchecked_return(FILE *file, long offset) {
  fseek(file, offset, SEEK_SET);  // diagnostic required
}
```

NOTE Return values from the following functions (Table 3) do not need to be checked because their historical use has overwhelmingly omitted error checking, and the consequences are not relevant to security.

Table 3 — Example library functions and returns

Function	Successful return	Error return
`printf`	number of characters (nonnegative)	negative
`putchar`	character written	`EOF`
`puts`	nonnegative	`EOF` (negative)
`putwchar`	wide character written	`WEOF`
`vprintf`	number of characters (nonnegative)	negative
`vwprintf`	number of wide characters (nonnegative)	negative
`wprintf`	number of wide characters (nonnegative)	negative

Exceptions

— EX1: The use of a `void` cast to signify programmer intent to ignore a return value from a function need not be diagnosed.

EXAMPLE This example shows an acceptable use of this exception.

```
void foo(FILE *file) {
  (void)fputs("foo", file);
  /* ... */
}
```

— EX2: Ignoring the return value of a function that cannot fail or whose return value cannot signify an error condition need not be diagnosed. For example, `strcpy` is one such function.

5.20 Forming invalid pointers by library function [libptr]

Rule

Invoking a C library function with an argument or arguments that causes the function to form a pointer that does not point into or just past the end of the object shall be diagnosed.

Rationale

Many C Standard Library functions manipulate individual objects or arrays of objects either one element at a time or one byte at a time. With a few exceptions, such functions typically take at least two arguments for each object (or array) they manipulate:

— a valid pointer into the object or storage for an object and

— an integer argument indicating how many elements or bytes of the object to manipulate.

C identifies the following undefined behavior:

UB	Description
109	The pointer passed to a library function array parameter does not have a value such that all address computations and object accesses are valid (7.1.4).

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

5.20.1 Library functions that take a pointer and an integer

The following standard library functions take a pointer argument and a size argument, with the constraint that the pointer must point to a valid memory object of at least the number of bytes or wide characters (as appropriate) indicated by the size argument.

fgets	fread	fwrite	mblen
memchr	memset	fgetws	wmemchr
wmemset	mbrlen	tmpnam_s	gets_s
getenv_s	memset_s	strerror_s	strnlen_s
asctime_s	ctime_s	wcscpy_s	wcsncpy_s
wmemcpy_s	wmemmove_s	wcscat_s	wcsncat_s
wcsnlen_s			

5.20.2 Library functions that take two pointers and an integer

The following standard library functions take two pointer arguments and a size argument, with the constraint that both pointers must point to valid memory objects of at least the number of bytes or wide characters as appropriate, indicated by the size argument.

mbtowc	wctomb	mbtowcs	wcstombs
memcpy	memmove	strncpy	strncat
memcmp	strncmp	strxfrm	mbrtoc16
mbrtoc32	wcsncpy	wmemcpy	wmemmove
wcsncat	wcsncmp	wcsxfrm	wmemcmp
mbrtowc	wcrtomb	mbsrtowcs	wcsrtombs
wctomb_s	mbtowcs_s	wcstombs_s	memcpy_s
memmove_s	strcpy_s	strncpy_s	strcat_s
strncat_s	wcscpy_s	wcsncpy_s	wmemcpy_s
wmemmove_s	wcscat_s	wcsncat_s	wcrtomb_s
mbsrtowcs_s	wcsrtombs_s		

5.20.3 Library functions that take a pointer and two integers

The following standard library functions take a pointer argument and two size arguments, with the constraint that the pointer must point to a valid memory object containing at least as many bytes as the product of the two size arguments.

bsearch

qsort

bsearch_s

qsort_s

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the size argument used in the `memset()` call is one longer than the size allocated for `p`.

```
void f1(size_t nchars) {
  char *p = (char *)malloc(nchars);
  const size_t n = nchars + 1;
  if (p) {
    memset(p, 0, n); // diagnostic required
    /* ... */
  }
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the size argument used in the `memset()` call is possibly miscalculated. There is no requirement that the size of an `int` is the same as the size of a `float`.

```
void f2(void) {
  float a[4];
  const size_t n = sizeof(int) * 4;
  void *p = a;

  memset(p, 0, n);  // diagnostic required

  /* ... */
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the size argument used in the `memcpy()` call is possibly miscalculated. The size of an `int` does not need to be smaller than that of a `double`.

```
void f3(int *a) {
  double b = 3.14;
  const size_t n = sizeof(*a);
  void *p = a;
  void *q = &b;

  if ((memcpy(p, q, n)) == p) {  // diagnostic required
    /* ... */
  }
  /* ... */
}
```

EXAMPLE 4 In this noncompliant example, a diagnostic is required because the value of `n` is not computed correctly, allowing a possible write past the end of the object referenced by `p`.

```
void f4(char p[], const char *q) {
  const size_t n = sizeof(p);
  if ((memcpy(p, q, n)) == p) {  // diagnostic required
    /* ... */
  }

  /* ... */
}
```

5.21 Allocating insufficient memory [insufmem]

Rule

A call to a standard memory allocation function is *presumed to be intended for type* `T *` when it appears in any of the following contexts.

— in the right operand of an assignment to an object of type `T *`, or

— in an initializer for an object of type `T *`, or

— in an expression that is passed as an argument of type `T *`, or

— in the expression of a `return` statement for a function returning type `T *`.

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

[AC1] A call to a standard memory allocation function taking a size integer argument `n` and presumed to be intended for type `T *` shall be regarded as an array of `N` elements, where `N = n / sizeof(T)`.

Any allocation where `N == 0` shall be diagnosed (i.e. where `n < sizeof(T)`). Also, any attempt to use this array in a manner that causes its array bound to be violated shall be diagnosed. [AC1]

The following are the standard memory allocation functions that take a size integer argument and return a pointer.

```
aligned_alloc

calloc

malloc

realloc
```

Rationale

Returning insufficient memory from a memory allocation function is likely to result in undefined behavior when that memory is accessed.

Example(s)

[AC1] EXAMPLE 1

```
struct S1 {
    unsigned int x;
    float y;
    struct S1 *z;
};
struct S1 *f1(void) {
    struct S1 *p = (struct S1*)malloc(sizeof(p)); // diagnostic required
    return p;
}
```

EXAMPLE 2

```
wchar_t *f2(void) {
    const wchar_t *p = L"Hello, World!";
    const size_t n = (wcslen(p) + 1);
    wchar_t *q = (wchar_t *)malloc(n);
    wcscpy(q, p); // diagnostic required
    return q;
}
```
[AC1]

5.22 Forming or using out-of-bounds pointers or array subscripts [invptr]

Rule

Using pointer arithmetic so that the result does not point into or just past the end of the same object, using invalid pointers in arithmetic expressions, or dereferencing pointers that do not point to a valid object shall be diagnosed.

Likewise, using an array subscript so that the resulting reference does not refer to an element in the array also shall be diagnosed.

Rationale

C identifies five distinct situations in which undefined behavior may arise as a result of invalid pointer operations:

© ISO/IEC 2013 – All rights reserved

UB	Description
46	Addition or subtraction of a pointer into, or just beyond, an array object and an integer type produces a result that does not point into, or just beyond, the same array object (6.5.6).
47	Addition or subtraction of a pointer into, or just beyond, an array object and an integer type produces a result that points just beyond the array object and is used as the operand of a unary * operator that is evaluated (6.5.6).
49	An array subscript is out of range, even if an object is apparently accessible with the given subscript (as in the lvalue expression a[1][7] given the declaration int a[4][5]) (6.5.6).
62	An attempt is made to access, or generate a pointer to just past, a flexible array member of a structure when the referenced object provides no elements for that array (6.7.2.1).
109	The pointer passed to a library function array parameter does not have a value such that all address computations and object accesses are valid (7.1.4).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because if `f` is called with a negative argument for `index`, an out-of-bounds pointer is formed.

```
#define TABLESIZE 100

static int table[TABLESIZE];

int *f(int index) {
  if (index < TABLESIZE) {
    return table + index;  // diagnostic required
  }

  return NULL;
}
```

EXAMPLE 2 In this compliant example, a diagnostic is not required because when the parameter `index` is negative, an out-of-bounds pointer cannot be returned.

```
#define TABLESIZE 100

static int table[TABLESIZE];

int *f(int index) {
  if (0 <= index && index < TABLESIZE) {
    return table + index;
  }

  return NULL;
}
```

EXAMPLE 3 In this compliant example, a diagnostic is not required because the parameter `index` cannot be negative and an out-of-bounds pointer cannot be returned.

```
#define TABLESIZE 100

static int table[TABLESIZE];

int *f(size_t index) {
  if (index < TABLESIZE) {
    return table + index;
  }

  return NULL;
}
```

EXAMPLE 4 In this noncompliant example, a diagnostic is required because if the string `path` does not contain the backslash character in the first `MAX_MACHINE_NAME_LENGTH + 1` characters, then `machine_name` will be dereferenced past the end pointer.

```
#define MAX_MACHINE_NAME_LENGTH 64
```


© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.


```
char *get_machine_name(const char *path) {
  char *machine_name = (char *)malloc(MAX_MACHINE_NAME_LENGTH + 1);
  if (machine_name == NULL) {
    return NULL;
  }

  while (*path != '\\') {
    *machine_name++ = *path++;  // diagnostic required
  }

  *machine_name = '\0';

  return machine_name;
}
```

EXAMPLE 5 In this compliant example, a diagnostic is not required because the string `path` is guaranteed to contain a backslash character within the first `MAX_MACHINE_NAME_LENGTH` characters when the string is copied to `machine_name`.

```
#define MAX_MACHINE_NAME_LENGTH 64

char *get_machine_name(const char *path) {
  const char *machine_name_end = strchr(path, '\\');
  if (machine_name_end == NULL
    || machine_name_end >= path + MAX_MACHINE_NAME_LENGTH) {
    return NULL;
  }

  char *machine_name = (char *)malloc(MAX_MACHINE_NAME_LENGTH + 1);
  if (machine_name == NULL) {
    return NULL;
  }

  const char *p = machine_name;

  while (path != p) {
    *p++ = *path++;
  }

  *p = '\0';

  return machine_name;
}
```

EXAMPLE 6 In this noncompliant example, a diagnostic is required because a value is stored beyond the end of the array `table` when the parameter `pos` equals the variable `size`.

```
static int *table = NULL;
static size_t size = 0;

int insert_in_table(size_t pos, int value) {
  if (pos > size) {
    int *tmp = (int *)realloc(table, sizeof(table[0]) * (pos + 1));
    if (tmp == NULL) {
      /* ... */
    }

    size = pos + 1;
    table = tmp;
  }

  table[pos] = value;  // diagnostic required
  return 0;
}
```

EXAMPLE 7 In this compliant example, a diagnostic is not required because a value is stored within the bounds of the array `table` when the parameter `pos` equals the variable `size`.

```
static int *table = NULL;
static size_t size = 0;
```

© ISO/IEC 2013 – All rights reserved

```
int insert_in_table(size_t pos, int value) {
  if (pos >= size) {
    int *tmp = (int *)realloc(table, sizeof(table[0]) * (pos + 1));
    if (tmp == NULL) {
      /* ... */
    }

    size  = pos + 1;
    table = tmp;

  }

  table[pos] = value;
  return 0;
}
```

EXAMPLE 8 In this noncompliant example, a diagnostic is required because a value is stored beyond the end of the arrays in `matrix[0]` through `matrix[4]` when `j` has values greater than 4.

```
enum { COLS = 5, ROWS = 7 };
static int matrix[ROWS][COLS];

void init_matrix(int x) {
  for (size_t i = 0; i != COLS; ++i) {
    for (size_t j = 0; j != ROWS; ++j) {
      matrix[i][j] = x;  // diagnostic required
    }
  }
}
```

EXAMPLE 9 In this compliant example, a diagnostic is not required because all values are stored within the bounds of the arrays in `matrix[0]` through `matrix[4]`.

```
enum { COLS = 5, ROWS = 7 };
static int matrix[ROWS][COLS];

void init_matrix(int x) {
  for (size_t i = 0; i != ROWS; ++i) {
    for (size_t j = 0; j != COLS; ++j) {
      matrix[i][j] = x;
    }
  }
}
```

EXAMPLE 10 In this noncompliant example, a diagnostic is required because the expression `first++` results in a pointer beyond the end of the array `buf` when `buf` contains no elements.

```
struct S {
  size_t len;
  char buf[];
};

char *find(struct S *s, int c) {
  char *first = s->buf;
  char *last = s->buf + s->len;

  while (first++ != last) {  // diagnostic required
    if (*first == (unsigned char)c) {
      return first;
    }
  }

  return NULL;
}

void g(void) {
  struct S *s = (struct S *)malloc(sizeof(struct S));
  if (s != NULL) {
    s->len = 0;
    /* ... */
    char *where = find(s, '.');
```


© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
    if (where == NULL) {
      return;
    }
  }

  /* ... */
}
```

EXAMPLE 11 In this compliant example, a diagnostic is not required because the expression `first++` does not occur unless `buf` contains elements.

```
struct S {
  size_t len;
  char buf[];
};

char *find(struct S *s, int c) {
  char *first = s->buf;
  char *last = s->buf + s->len;

  while (first != last) {
    if (*first++ == (unsigned char)c) {
      return first;
    }
  }

  return NULL;
}

void g(void) {
  struct S *s = (struct S *)malloc(sizeof(struct S));
  if (s) {
    s->len = 0;
    /* ... */
    char *where = find(s, '.');
    if (where == NULL) {
      return;
    }
  }

  /* ... */
}
```

EXAMPLE 12 In this noncompliant example, a diagnostic is required because the expression `buf[strlen(buf) - 1]` assumes that the first byte of the parameter to `fgets`, `buf`, is non-null.

```
void f(void) {
  char buf[BUFSIZ];

  if (fgets(buf, sizeof(buf), stdin)) {
    buf[strlen(buf) - 1] = '\0';  // diagnostic required
    puts(buf);
  }
}
```

EXAMPLE 13 In this noncompliant example, a diagnostic is required because the integer `skip` is scaled when added to the pointer `s` and may point outside the bounds of the object referenced by `s`.

```
struct big {
  unsigned long long ull_1;
  unsigned long long ull_2;
  unsigned long long ull_3;
  int si_4;
  int si_5;
};

void g(void) {
  size_t skip = offsetof(struct big, ull_2);
  struct big *s = (struct big *)malloc(4 * sizeof(struct big));
  if (!s) {
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
    /* ... */
  }

  memset(s + skip, 0, sizeof(struct big) - skip);  // diagnostic required

  /* ... */
}
```

5.23 Freeing memory multiple times [dblfree]

Rule

Freeing memory multiple times shall be diagnosed.

Rationale

Freeing memory multiple times results in *double-free* vulnerabilities.

C identifies the following undefined behavior:

UB	Description
179	The pointer argument to the `free` or `realloc` function does not match a pointer earlier returned by a memory management function, or the space has been deallocated by a call to `free` or `realloc` (7.22.3.3, 7.22.3.5).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because `x` could be freed twice depending on the value of `error_condition`.

```
void f(size_t num_elem) {
  int error_condition = 0;

  int *x = (int *)malloc(num_elem * sizeof(int));
  if (x == NULL) {
    /* ... */
  }
  /* ... */
  if (error_condition == 1) {
    /* ... */
    free(x);
  }
  /* ... */
  free(x);  // diagnostic required
  x = NULL;
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because `realloc` may free `c_str1` when it returns `NULL`, resulting in `c_str1` being freed twice.

```
void g(char *c_str1, size_t size) {
  char *c_str2 = (char *)realloc(c_str1, size);
  if (c_str2 == NULL) {
    free(c_str1);  // diagnostic required
    return;
  }
}
```

According to ISO/IEC 9899:2011, 7.22.3, paragraph 1,

> *If the size of the space requested is zero, the behavior is implementation-defined: either a null pointer is returned, or the behavior is as if the size were some nonzero value, except that the returned pointer shall not be used to access an object.*

And according to ISO/IEC 9899:2011, 7.22.3.5, paragraph 3,

> *If memory for the new object cannot be allocated, the old object is not deallocated and its value is unchanged.*

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

If `realloc` is called with `size` equal to 0, then if a `NULL` pointer is returned, the old value should be unchanged. However, there are some common but nonconforming implementations that free the pointer, which means that calling `free` on the original pointer might result in a double-free vulnerability. However, not calling `free` on the original pointer might result in a memory leak.

Exception

Some library implementations accept and ignore a deallocation of already-free memory. If all libraries used by a project have been validated as having this behavior, then this violation does not need to be diagnosed.

5.24 Including tainted or out-of-domain input in a format string [usrfmt]

Rule

Invoking any of the formatted input/output functions identified in ISO/IEC 9899:2011, 7.21.6, where the format argument references string data that is tainted (4.15) or out-of-domain with respect to character content, shall be diagnosed.

An empty string is not tainted.

NOTE Any comparison of a character in the string to a value other than the null character may be considered to sanitize the string.

Rationale

Invoking a formatted input/output function where the format argument references string data that is tainted (4.15) or out-of-domain with respect to character content can result in undefined or unexpected behavior.

An attacker who can fully or partially control the contents of a format string can crash a vulnerable process, view the contents of the stack, view memory content, or write to an arbitrary memory location and consequently execute arbitrary code with the permissions of the vulnerable process.[3]

Formatted output functions are particularly dangerous because many programmers are unaware of their capabilities. (For example, they can write an integer value to a specified address using the `%n` conversion specifier.)

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because a format string is read from a tainted source and passed as an argument to the `vfprintf` function.

```
void format_error(const char *filename, ...) {
  FILE *fd = fopen(filename, "r");
  if (fd == NULL) {
    /* ... */
  }

  char fmt[BUFSIZ];
  if (fgets(fmt, BUFSIZ, fd) == NULL) {
    /* ... */
  }

  va_list va;
  va_start(va, filename);
  vfprintf(stderr, fmt, va);  // diagnostic required
  va_end(va);

  fclose(fd);
}
```

EXAMPLE 2 In this compliant example, a diagnostic is not required because the format string that is read from a tainted source and passed as an argument to the `vfprintf` function is first sanitized.

```
void safe_format_error(const char *filename, ...) {
  FILE *fd = fopen(filename, "r");
  if (fd == NULL) {
```

© ISO/IEC 2013 – All rights reserved

```
    /* ... */
  }

  char fmt[BUFSIZ];
  if (fgets(fmt, BUFSIZ, fd) == NULL) {
    /* ... */
  }

  /* only allow %d in the format string: */
  const char *fc;
  for (fc = fmt; *fc != '\0'; ++fc) {
    if (*fc == '%' && (fc[1] != '%' && fc[1] != 'd')) {
      fclose(fd);
      return;
    }
  }

  va_list va;
  va_start(va, filename);
  vfprintf(stderr, fmt, va);
  va_end(va);

  fclose(fd);
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the string `user` may contain a tainted value.

```
int incorrect_password(void) {
  int ret;
  char user[BUFSIZ];

  GET_TAINTED_STRING(user, BUFSIZ);

  static const char MSG_FORMAT[] = "%s cannot be authenticated.\n";

  size_t size = strlen(user) + sizeof(MSG_FORMAT);
  char *msg = (char *)malloc(size);

  if (msg == NULL) {
    fprintf(stderr, "Could not malloc memmory in %s\n", __func__);
    return -1;
  } else {
    ret = snprintf(msg, size, MSG_FORMAT, user); // MSG_FORMAT is not tainted

    if (ret < 0) {
      fprintf(stderr, "snprintf encoding error occurred in %s\n", __func__);
    } else if (ret >= size) {
      printf("snprintf returned %d in %s\n", ret, __func__);
    }

    fprintf(stderr, msg); // diagnostic required

    free(msg);
  }

 return 0;

}
```

EXAMPLE 4 In this compliant example, a diagnostic is not required because the argument `fmt` is constrained to be one of the elements of the `formats` array, none of which are tainted values.

```
enum int_tag { I_char, I_shrt, I_int, I_long, I_llong };
static const char *const formats[] = { "%hhi", "%hi", "%i", "%li", "%lli" };

static int fmtintv(enum int_tag tag, const char *fmt, va_list va) {
  return vfprintf(stdout, fmt, va);
}
```

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.


```
int format_integer(enum int_tag tag, ...) {
  va_list va;
  int n;
  if (tag < I_char || I_llong < tag)
    return -1;
  va_start(va, tag);
  n = fmtintv(tag, formats[tag], va);
  va_end(va);
  return n;
}
```

5.25 Incorrectly setting and using `errno` [inverrno]

Rule

Incorrectly setting and using `errno` shall be diagnosed.

The correct way to set and check `errno` is defined in the following cases.

5.25.1 Library functions that set `errno` and return an in-band error indicator

A program that uses `errno` for error checking shall set `errno` to zero before calling one of these library functions, and then it shall inspect `errno` before a subsequent library function call.

The functions in Table 4 set `errno` and return an in-band error indicator.

Table 4 — Functions that set `errno` and return an in-band error indicator

Function name	Return value	errno value
`ftell`	`-1L`	positive
`stroumax`	`UINTMAX_MAX`	`ERANGE`
`strtod`,[a] `wcstod`	`zero` or ±`HUGE_VAL`	`ERANGE`
`strtof, wcstof`	`zero` or ±`HUGE_VALF`	`ERANGE`
`strtoimax`	`INTMAX_MIN` or `INTMAX_MAX`	`ERANGE`
`strtol, wcstol`	`LONG_MIN` or `LONG_MAX`	`ERANGE`
`strtold, wcstold`	`zero` or ±`HUGE_VALL`	`ERANGE`
`strtoll, wcstoll`	`LLONG_MIN` or `LLONG_MAX`	`ERANGE`
`strtoul, wcstoul`	`ULONG_MAX`	`ERANGE`
`strtoull, wcstoull`	`ULLONG_MAX`	`ERANGE`
`wcstoimax`	`INTMAX_MIN` or `INTMAX_MAX`	`ERANGE`
`wcstoumax`	`UINTMAX_MAX`	`ERANGE`

[a] However, according to the C Standard, if the result of `strtod`, `strtof`, or `strtold` (and the related wide character functions) underflows, "the functions return a value whose magnitude is no greater than the smallest normalized positive number in the return type; whether `errno` acquires the value `ERANGE` is implementation-defined."

5.25.2 Library functions that set `errno` and return an out-of-band error indicator

A program may use `errno` for error checking after calling one of these functions without setting `errno` to zero before calling the function. Then the program shall inspect `errno` conditional on the return value and before a subsequent library function call.

The functions in Table 5 set `errno` and return an out-of-band error indicator.

© ISO/IEC 2013 – All rights reserved

Table 5 — Library functions that set `errno` value and return an out-of-band error indicator

Function name	Return value	errno value
`fgetpos`	nonzero	positive
`fsetpos`	nonzero	positive
`mbrtowc`	`(size_t)(-1)`	`EILSEQ`
`mbsrtowcs`	`(size_t)(-1)`	`EILSEQ`
`signal`[a]	`SIG_ERR`	`positive`
`wcrtomb`	`(size_t)(-1)`	`EILSEQ`
`wcsrtombs`	`(size_t)(-1)`	`EILSEQ`

[a] The value of `errno` is indeterminate if signal returns `SIG_ERR` from within a signal handler that was triggered by a signal that occurred other than as the result of a call to `abort` or `raise`.

5.25.3 Library functions that occasionally set `errno` and return an out-of-band error indicator

The `fgetwc` and `fputwc` functions return `WEOF` in multiple cases, only one of which results in setting `errno`. Therefore, the program shall set `errno` to zero before calling these functions.

The functions in Table 6 set `errno` and return an out-of-band error indicator.

Table 6 — Library functions that occasionally set `errno` value and return an out-of-band error indicator

Function name	Return value	`errno` value
`fgetwc`	`WEOF`	`EILSEQ`
`fputwc`	`WEOF`	`EILSEQ`

5.25.4 Library functions that may or may not set `errno`

Programs shall not rely on `errno` after calling a function that might set `errno` when an error occurs because the function might have altered `errno` in an implementation-defined way.

The functions defined in `<complex.h>` could or could not set `errno` when an error occurs.

The functions defined in `<math.h>` set `errno` in the following conditions:

— If there is a domain error and the integer expression `math_errhandling & MATH_ERRNO` is nonzero, then `errno` is set to `EDOM`.

— According to ISO/IEC 9899:2011, 7.12.1, paragraph 5, "If a floating result overflows and default rounding is in effect, then the function returns the value of the macro `HUGE_VAL`, `HUGE_VALF`, or `HUGE_VALL` according to the return type, with the same sign as the correct value of the function; if the integer expression `math_errhandling & MATH_ERRNO` is nonzero, the integer expression `errno` acquires the value `ERANGE`."

— Similarly, according to ISO/IEC 9899:2011, 7.12.1, paragraph 6, "The result underflows if the magnitude of the mathematical result is so small that the mathematical result cannot be represented, without extraordinary roundoff error, in an object of the specified type. If the result underflows, the function returns an implementation-defined value whose magnitude is no greater than the smallest normalized positive number in the specified type; if the integer expression `math_errhandling & MATH_ERRNO` is nonzero, whether `errno` acquires the value `ERANGE` is implementation-defined."

The functions `atof`, `atoi`, `atol`, and `atoll` may or may not set `errno` when an error occurs.

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

5.25.5 Library functions that do not explicitly set `errno`

Programs shall not rely on `errno` to determine whether an error occurred after calling a Standard C Library function that does not explicitly set `errno`. Such a function may set `errno` even when no error has occurred. All library functions that have not been discussed yet are functions that do not explicitly set `errno`.

Rationale

Incorrectly setting and using `errno` can result in undefined or unexpected behavior.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because `errno` is used for error checking and `errno` is not set to zero before the C Standard Library function `strtoul` is called.

```
void f(const char *c_str) {
  char *endptr = NULL;
  unsigned long number = strtoul(c_str, &endptr, 0);

  if (endptr == c_str
  || (number == ULONG_MAX && errno == ERANGE)) {  // diagnostic required
    /* ... */
  } else {
    /* ... */
  }

  /* ... */
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because `errno` is used for error checking and the return value of the call to the C Standard Library function `signal` is not checked before checking `errno`.

```
void g(void) {
  errno = 0;
  signal(SIGINT, SIG_DFL);
  if (errno != 0) {  // diagnostic required
    /* ... */
  }
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because `errno` is used for error checking and `errno` is checked after the call to the C Standard Library function `setlocale` because `setlocale` does not explicitly set `errno`.

```
void h(void) {
  errno = 0;
  setlocale(LC_ALL, "");
  if (errno != 0) {  // diagnostic required
    /* ... */
  }
}
```

5.26 Integer division errors [diverr]

Rule

Integer values that are used as operands to the `/` or `%` operators where the second operand to the `/` operator or the `%` operator is tainted shall be diagnosed.

Tainted values that are used as either operand to the `/` operator or the `%` operator shall be diagnosed if the quotient of the two operands is not representable.

Rationale

© ISO/IEC 2013 – All rights reserved

C identifies two conditions under which division and remainder operations result in undefined behavior:

UB	Description
45	The value of the second operand of the `/` or `%` operator is zero (6.5.5).
n/a	If the quotient `a/b` is not representable, the behavior of both `a/b` and `a%b` is undefined (6.5.5).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the expression `x / y` can result in a divide-by-zero error or in a quotient that is not representable.

```
int divide(int x) {
  int y;
  GET_TAINTED_INTEGER(int, y);

  return x / y;  // diagnostic required
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the expression `x % y` can result in a divide-by-zero error or a quotient of the two operands that is not representable.

```
int remainder(int x) {
  int y;
  GET_TAINTED_INTEGER(int, y);

  return x % y;  // diagnostic required
}
```

EXAMPLE 3 In this compliant example, the expression `x / y` cannot result in a divide-by-zero error or in a quotient that is not representable.

```
int divide(int x) {
  int y;
  GET_TAINTED_INTEGER(int, y);
  if ( (y == 0) || ( (x == INT_MIN) && (y == -1) ) ) {
     /* Handle error */
  }
  else {
    return x / y;
  }
}
```

EXAMPLE 4 In this compliant example, the expression `x % y` cannot result in a divide-by-zero error or in a quotient that is not representable.

```
int remainder(int x) {
  int y;
  GET_TAINTED_INTEGER(int, y);
  if ( (y == 0) || ( (x == INT_MIN) && (y == -1) ) ) {
     /* Handle error */
  }
  else {
    return x % y;
  }
}
```

5.27 Interleaving stream inputs and outputs without a flush or positioning call [ioileave]

Rule

The following scenarios shall be diagnosed:

— receiving input from a stream directly following an output to that stream without an intervening call to `fflush`, `fseek`, `fsetpos`, or `rewind`, if the file is not at end-of-file or

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

— outputting to a stream after receiving input from that stream without a call to `fseek`, `fsetpos`, or `rewind`, if the file is not at end-of-file

Rationale

C identifies the following undefined behavior:

UB	Description
151	An output operation on an update stream is followed by an input operation without an intervening call to the `fflush` function or a file positioning function, or an input operation on an update stream is followed by an output operation with an intervening call to a file positioning function (7.21.5.3).

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because `fread` and `fwrite` are called on the same file without an intervening call to `fflush`, `fseek`, `fsetpos`, or `rewind` on the file.

```
void f(const char *filename, char append_data[BUFSIZ]) {
  char data[BUFSIZ];
  FILE *file;

  file = fopen(filename, "a+");
  if (file == NULL) {
    /* ... */
  }

  if (fwrite(append_data, sizeof(char), BUFSIZ, file) != BUFSIZ) {
    /* ... */
  }

  if (fread(data, sizeof(char), BUFSIZ, file) != 0) {  // diagnostic required
    /* ... */
  }

  fclose(file);
}
```

5.28 Modifying string literals [strmod]

Rule

Directly modifying any portion of a string literal, assigning a string literal to a pointer to non-`const`, or casting a string literal to a pointer to non-`const` shall be diagnosed. For the purposes of this rule, the returned value of the library functions `strpbrk`, `strchr`, `strrchr`, `wcspbrk`, `wcschr`, and `wcsrchr` shall be treated as a string literal if the first argument is a string literal. For the purposes of this rule, a pointer to (or array of) `const` characters shall be treated as a string literal.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the string literal "`string literal`" is modified through the pointer `p`.

```
void f1(void) {
  char *p  = "string literal";
  p[0] = 'S';  // diagnostic required
  /* ... */
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the string literal "`/tmp/edXXXXXX`" is modified by the C Standard Library function `tmpnam`.

```
void f2(void) {
  if (tmpnam("/tmp/edXXXXXX")) {  // diagnostic required
    /* ... */
  }
```

© ISO/IEC 2013 – All rights reserved

```
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the string literal "/tmp/filename" is modified through the pointer returned from the C Standard Library function strrchr.

```
void f3(void) {
  char *last_slash = strrchr("/tmp/filename", '/');
  *last_slash = '\0';  // diagnostic required
  /* ... */
}
```

EXAMPLE 4 In this noncompliant example, a diagnostic is required because the string literal "/tmp/filename" is modified through the pointer returned from the C Standard Library function strrchr.

```
void f4(void) {
  *strrchr("/tmp/filename", '/') = '\0';  // diagnostic required
  /* ... */
}
```

EXAMPLE 5 In this noncompliant example, a diagnostic is required because the string literal "/tmp/filename" is modified.

```
void f5(void) {
  "/tmp/filename"[4] = '\0';  // diagnostic required
  /* ... */
}
```

Exception

No diagnostic need be issued if the analyzer can determine that the value of the pointer to non-const is never used to attempt to modify the characters of the string literal.

EXAMPLE

```
int main(void) {
  char *p = "abc";
  printf("%s\n", p);
  return EXIT_SUCCESS;
}
```

5.29 Modifying the string returned by getenv, localeconv, setlocale, and strerror [libmod]

Rule

Modifying the objects or strings returned by the library functions listed in getenv, localeconv, setlocale, and strerror shall be diagnosed.

Rationale

C identifies the following three instances of undefined behavior, which arise as a result of modifying the data structures or strings returned from getenv, localeconv, setlocale, and strerror:

UB	Description
120	The program modifies the string pointed to by the value returned by the setlocale function (7.11.1.1).
121	The program modifies the structure pointed to by the value returned by the localeconv function (7.11.2.1).
184	The string set up by the getenv or strerror function is modified by the program (7.22.4.6, 7.24.6.2).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the string returned from the C Standard Library function setlocale is modified.

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
void f1(void) {
  char *locale = setlocale(LC_ALL, 0);
  if (locale != NULL) {
    char *cats[8];
    char *sep = locale;
    cats[0] = locale;
    int i;

    if (sep) {
      for (i = 0; (sep = strstr(sep, ";:")) && i < 8; ++i) {
        *sep = '\0';  // diagnostic required
        cats[i] = ++sep;
      }
    }
  }

  /* ... */
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the object returned from the C Standard Library function `localeconv` is modified.

```
void f2(void) {
  struct lconv *conv = localeconv();

  if ('\0' == conv->decimal_point[0]) {
    conv->decimal_point = ".";  // diagnostic required
  }

  if ('\0' == conv->thousands_sep[0]) {
    conv->thousands_sep = ",";  // diagnostic required
  }

  /* ... */
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the string returned from the C Standard Library function `getenv` is modified.

```
void f3(void) {
  char *shell_dir = getenv("SHELL");

  if (shell_dir != NULL) {
    char *slash = strrchr(shell_dir, '/');
    if (slash) {
      *slash = '\0';  // diagnostic required
    }

    /* use shell_dir */
  }
}
```

EXAMPLE 4 In this noncompliant example, a diagnostic is required because the string returned from the C Standard Library function `strerror` is modified.

```
const char *f4(int error) {
  char buf[BUFFER_SIZE];
  sprintf(buf, "(errno = %d)", error);

  char *error_str = strerror(error);

  strcat(error_str, buf);  // diagnostic required
  return error_str;
}
```

5.30 Overflowing signed integers [intoflow]

Rule

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Whenever at least one operand is tainted, signed integer operations that trap and that can overflow shall be diagnosed.

Rationale

Signed integer overflow is undefined behavior.

UB	Description
36	An exceptional condition occurs during the evaluation of an expression (6.5).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required on implementations that trap on signed integer overflow because the expression `x + 1` may result in signed integer overflow.

```
int add(void) {
  int x;
  GET_TAINTED_INTEGER(int, x);

  return x + 1;  // diagnostic required
}
```

EXAMPLE 2 In this compliant example, a diagnostic is not required because the expression `x + 1` cannot result in signed integer overflow.

```
int add(void) {
  int x;
  GET_TAINTED_INTEGER(int, x);

  if (x < INT_MAX) {
    return x + 1;
  } else {
    return INT_MIN;
  }
}
```

5.31 Passing a non-null-terminated character sequence to a library function that expects a string [nonnullcs]

Rule

Passing a character sequence or wide character sequence that is not null-terminated to a library function that expects a string or wide string argument shall be diagnosed.

Rationale

Many library functions accept a string or wide string argument with the constraint that the string they receive is properly null-terminated. Passing a character sequence or wide character sequence that is not null-terminated to such a function can result in accessing memory that is outside the bounds of the object.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the character sequence `str` will not be null-terminated when passed as an argument to `printf`.

```
char str[3] = "abc";
printf("%s\n", str);              // diagnostic required
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the wide character sequence `cur_msg` will not be null-terminated when passed to `wcslen`. This will occur if `lessen_memory_usage` is invoked while `cur_msg_size` still has its initial value of 1024.

```
wchar_t *cur_msg = NULL;
size_t cur_msg_size = 1024;
size_t cur_msg_len = 0;
```


© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
void lessen_memory_usage(void) {
  wchar_t *temp;
  size_t temp_size;

  /* ... */

  if (cur_msg != NULL) {
    temp_size = cur_msg_size / 2 + 1;
    temp = realloc(cur_msg, temp_size * sizeof(wchar_t));
           // temp & cur_msg might not be null-terminated
    if (temp == NULL) {
      /* ... */
    }

    cur_msg = temp;
    cur_msg_size = temp_size;
    cur_msg_len = wcslen(cur_msg);  // diagnostic required
  }
}
```

EXAMPLE 3 In this compliant example, a diagnostic is not required because `cur_msg` will always be null-terminated when passed to `wcslen`.

```
wchar_t *cur_msg = NULL;
size_t cur_msg_size = 1024;
size_t cur_msg_len = 0;

void lessen_memory_usage(void) {
  wchar_t *temp;
  size_t temp_size;

  /* ... */

  if (cur_msg != NULL) {
    temp_size = cur_msg_size / 2 + 1;
    temp = realloc(cur_msg, temp_size * sizeof(wchar_t));
           // temp & cur_msg might not be null-terminated
    if (temp == NULL) {
      /* ... */
    }

    cur_msg = temp;
    cur_msg[ temp_size - 1] = L'\0'; // cur_msg now properly null-terminated
    cur_msg_size = temp_size;
    cur_msg_len = wcslen(cur_msg);  // diagnostic not required
  }
}
```

5.32 Passing arguments to character-handling functions that are not representable as `unsigned char` [chrsgnext]

Rule

Arguments to the character-handling functions in `<ctype.h>` that are not representable as `unsigned char` shall be diagnosed.

The following character classification functions are affected:

isalnum	isalpha	isascii	isblank
iscntrl	isdigit	isgraph	islower
isprint	ispunct	isspace	isupper
isxdigit	toascii	toupper	tolower

Rationale

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

These character classification functions are defined only for values representable as `unsigned char` and the macro `EOF`.

Example(s)

EXAMPLE On implementations where plain `char` is signed, a diagnostic is required because the parameter to `isspace`, `*t`, is defined as a `const char *`, and this value may not be representable as an `unsigned char`.

```
size_t count_preceding_whitespace(const char *s) {
  const char *t = s;
  size_t length = strlen(s) + 1;

  while (isspace(*t) && (t - s < length)) {  // diagnostic required
    ++t;
  }
  return t - s;
}
```

5.33 Passing pointers into the same object as arguments to different `restrict`-qualified parameters [restrict]

Rule

Function arguments that are `restrict`-qualified pointers and reference overlapping objects shall be diagnosed.

Rationale

C identifies the following undefined behavior:

UB	Description
68	An object which has been modified is accessed through a restrict-qualified pointer to a const-qualified type, or through a restrict-qualified pointer and another pointer that are not both based on the same object (6.7.3.1).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the `restrict`-qualified pointer parameters to `memcpy`, `ptr1`, and `ptr2` reference overlapping objects.

```
void abcabc(void) {
  char c_str[]= "abc123edf";
  char *ptr1 = c_str;
  char *ptr2 = c_str + strlen("abc");

  memcpy(ptr2, ptr1, 6);  // diagnostic required
  puts(c_str);
}
```

EXAMPLE 2 In this noncompliant example, the `src` operand is used twice to refer to unmodified memory, which is allowed by the C Standard; the aliasing restrictions apply only when the object is modified. A diagnostic is required nontheless because the pointer `src` is twice a `restrict`-qualified pointer parameter to `dual_memcpy`, referencing overlapping objects.

```
void *dual_memcpy(
  void *restrict  s1, const void *restrict  s2, size_t n1,
  void *restrict  s3, const void *restrict  s4, size_t n2
) {
  memcpy(s1, s2, n1);
  memcpy(s3, s4, n2);

  return s1;
}

void f(void) {
  char dest1[10];
```

 © ISO/IEC 2013 – All rights reserved

```
  char dest2[10];
  char src[] = "hello";

  dual_memcpy(dest1, src, sizeof(src),
              dest2, src, sizeof(src));  // diagnostic required
  puts(dest1);
  puts(dest2);
}
```

5.34 Reallocating or freeing memory that was not dynamically allocated [xfree]

Rule

Calling `realloc` or `free` in cases where the `ptr` argument to either function may refer to memory that was not dynamically allocated shall be diagnosed.

Rationale

C identifies the following undefined behavior:

UB	Description
179	The pointer argument to the `free` or `realloc` function does not match a pointer earlier returned by a memory management function, or the space has been deallocated by a call to `free` or `realloc` (7.22.3.3, 7.22.3.5).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the pointer parameter to `realloc`, buf, does not refer to dynamically allocated memory.

```
#define BUFSIZE 256

void f(void) {
  char buf[BUFSIZE];
  char *p;
  /* ... */
  p = (char *)realloc(buf, 2 * BUFSIZE);  // diagnostic required
  /* ... */
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the pointer parameter to `free`, `c_str` may not refer to dynamically allocated memory.

```
#define MAX_ALLOCATION 1000

int main(int argc, char *argv[]) {
  char *c_str = NULL;
  size_t len;

  if (argc == 2) {
    len = strlen(argv[1]) + 1;
    if (len > MAX_ALLOCATION) {
      /* Handle error */
    }
    c_str = (char *)malloc(len);
    if (c_str == NULL) {
      /* Handle allocation error */
    }
    strcpy(c_str, argv[1]);
  }
  else {
    c_str = "usage: $>a.exe[string]";
    printf("%s\n", c_str);
  }
  /* ... */
  free(c_str);  // diagnostic required
  return EXIT_SUCCESS;
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
}
```

Exception

Some library implementations accept and ignore a deallocation of non-allocated memory (or, alternatively, cause a runtime-constraint violation). If all libraries used by a project have been validated as having this behavior, then this violation does not need to be diagnosed.

5.35 Referencing uninitialized memory [uninitref]

Rule

Accessing uninitialized memory by an lvalue of a type other than `unsigned char` shall be diagnosed.

Rationale

There are two main sources of uninitialized memory:

— uninitialized automatic variables and

— uninitialized memory returned by the memory management functions `malloc`, `realloc`, and `aligned_alloc`.

Uninitialized memory has indeterminate value, which for objects of some types can be a trap representation. Accessing uninitialized memory by an lvalue of a type other than `unsigned char` has undefined behavior. Typical consequences of accessing uninitialized memory relevant to security range from denial of service to information exposure as a result of leaking sensitive data previously stored in a memory region.

C identifies the following undefined behaviors:

UB	Description
11	The value of an object with automatic storage duration is used while it is indeterminate (6.2.4, 6.7.9, 6.8).
12	A trap representation is read by an lvalue expression that does not have character type (6.2.6.1).

It should be noted that while it is safe to copy a region of uninitialized storage into another location using a function such as `memcpy`, after the copy, the destination region has the same "uninitialized" contents as the source region even if it had been initialized to a determinate value before the copy.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the variable `sign` may be uninitialized when it is accessed in the `return` statement of the function `is_negative`.

```
void get_sign(int number, int *sign) {
  if (sign == NULL) {
    /* ... */
  }

  if (number > 0) {
    *sign = 1;
  } else if (number < 0) {
    *sign = -1;
  } // If number == 0, sign is not changed.
}

int is_negative(int number) {
  int sign;
  get_sign(number, &sign);

  return (sign < 0);  // diagnostic required
}
```


© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the variable `error_log` is uninitialized when it is passed to `sprintf`.

```
int do_auth(void) {
  int result = -1;

  /* ... */
  return result;
}

void report_error(const char *msg) {
  const char *error_log;
  char buffer[24];

  sprintf(buffer, "Error: %s", error_log);  // diagnostic required
  printf("%s\n", buffer);
}

int main(void) {
  if (do_auth() == -1) {
    report_error("Unable to login");
  }

  return EXIT_SUCCESS;
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the elements of the array `a` are uninitialized when they are accessed in the `for` loop.

```
void f(size_t n) {
  int *a = (int *)malloc(n * sizeof(int));
  if (a != NULL) {
    for (size_t i = 0; i != n; ++i) {
      a[i] = a[i] ^ a[i];  // diagnostic required
    }

    /* ... */
    free(a);
  }
}
```

EXAMPLE 4 In this noncompliant example, a diagnostic is required because the array elements `a[n..2n]` are uninitialized when they are accessed in the `for` loop.

```
void g(double *a, size_t n) {
  a = (double *)realloc(a, (n * 2 + 1) * sizeof(double));
  if (a != NULL) {
    for (size_t i = 0; i != n * 2 + 1; ++i) {
      if (a[i] < 0) {
        a[i] = -a[i];  // diagnostic required
      }
    }

    /* ... */
    free(a);
  }
}
```

5.36 Subtracting or comparing two pointers that do not refer to the same array [ptrobj]

Rule

Subtracting or relationally comparing two pointers that do not refer to the same array object, or one element past the same array object, shall be diagnosed. The relational operators are >, <, >=, and <=.

Rationale

C identifies two distinct situations in which undefined behavior may arise as a result of using pointers that do not point to the same object:

UB	Description
48	Pointers that do not point into, or just beyond, the same array object are subtracted (6.5.6).
53	Pointers that do not point to the same aggregate or union (nor just beyond the same array object) are compared using relational operators (6.5.8).

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the pointers `c_str` and `(char **)next_num_ptr` are subtracted and do not refer to the same array.

```
#define SIZE 256

void f(void) {
  int nums[SIZE];
  char *c_str[SIZE];
  int *next_num_ptr = nums;
  int free_bytes;

  /* ... */
  /* increment next_num_ptr as array fills */

  free_bytes = c_str - (char **)next_num_ptr;  // diagnostic required
  /* ... */
}
```

Exceptions

— EX1: Comparing two pointers within the same object does not need to be diagnosed.

— EX2: Subtracting two pointers to `char` within the same object does not need to be diagnosed.

5.37 Tainted strings are passed to a string copying function [taintstrcpy]

Rule

Tainted values that are referenced by the source argument to the `strcpy`, `strcat`, `wcscpy`, or `wcscat` function and that exceed the size of the destination array shall be diagnosed.

Rationale

Passing wide or narrow strings as the source argument to the `strcpy`, `strcat`, `wcscpy`, or `wcscat` function that exceed the size of the destination array can result in writing to memory that is outside the bounds of existing objects.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the size of the string referenced by `argv[0]` might be greater than the size of the destination array `pgm`.

```
void main(int argc, char *argv[]) {
  char pgm[BUFSIZ];

  if (argc > 1) {
    strcpy(pgm, argv[0]);  // diagnostic required
  }
}
```

5.38 Taking the size of a pointer to determine the size of the pointed-to type [sizeofptr]

Rule

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Using the `sizeof` operator on an array parameter shall be diagnosed.

Rationale

Using the `sizeof` operator on an array parameter frequently indicates a programmer error and can result in unexpected behavior.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the `sizeof` operator is applied to the pointer parameter `array`.

```
void clear(int array[]) {
  for (size_t i = 0;
       i < sizeof(array) / sizeof(array[0]); // diagnostic required
       ++i) {
     array[i] = 0;
  }
}
```

5.39 Using a tainted value as an argument to an unprototyped function pointer [taintnoproto]

Rule

Passing a tainted value as an argument to a call through a pointer to a function that was declared without a prototype shall be diagnosed.

Rationale

A pointer to a function declared without a prototype may refer to a function whose parameters ultimately flow into a restricted sink. When a prototype is available, an analyzer might be able to determine if the pointer points to such a function.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the tainted argument `tainted` is passed as an argument to a call through an unprototyped pointer to function `pf`. The initialization of `pf` and the definition of `restricted_sink` are informational and not necessary for this diagnosis.

```
void restricted_sink(int i) {
  int array[2];
  array[i] = 0;
}

void (*pf)() = restricted_sink;

void f(void) {
  int tainted;
  GET_TAINTED_INTEGER(int, tainted);
  (*pf)(tainted);  // diagnostic required
}
```

EXAMPLE 2 In this compliant example, a diagnostic is not required because the tainted argument `tainted2` is passed as an argument to a call through a properly prototyped pointer to function `pf2`.

```
void (*pf2)(int);

void g(void) {
  int tainted2;
  GET_TAINTED(int, tainted2);
  (*pf2)(tainted2);
}
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

5.40 Using a tainted value to write to an object using a formatted input or output function [taintformatio]

Rule

Calls to the `fscanf`, `scanf`, `vfscanf`, and `vscanf` functions that pass tainted values as arguments and that can result in writes outside the bounds of the specified object shall be diagnosed.

Calls to the `sscanf` and `vsscanf` functions that can result in writes outside the bounds of the specified object shall also be diagnosed when the input string is tainted.

Calls to the `sprintf` function that can result in writes outside the bounds of the destination array shall be diagnosed when any of its variadic arguments are tainted.

Rationale

Calls to the standard C formatted input functions declared in `<stdio.h>`, as well as to the `sprintf` function, can corrupt memory.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the call to `fscanf` can result in a write outside the character array `buf`.

```
char buf[BUF_LENGTH];
fscanf(stdin, "%s", buf); // diagnostic required
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the `sprintf` function will write outside the bounds of the character array `buf`.

```
int rc = 0;
int x;
GET_TAINTED_INTEGER(int, x);
char buf[sizeof("999")];
rc = sprintf(buf, "%d", x); // diagnostic required
if (rc == -1 || rc >= sizeof(buf)) {
  /* handle error */
}
```

5.41 Using a value for `fsetpos` other than a value returned from `fgetpos` [xfilepos]

Rule

Using an offset value for `fsetpos`, other than a value returned from `fgetpos`, shall be diagnosed.

Rationale

C identifies the following undefined behavior:

UB	Description
175	The `fsetpos` function is called to set a position that was not returned by a previous successful call to the `fgetpos` function on a stream associated with the same file (7.21.9.3).

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because an offset value other than one returned from `fgetpos` is used in a call to `fsetpos`.

```
FILE *opener(const char *filename) {
  fpos_t offset;

  if (filename == NULL) {
    /* ... */
  }
```


© ISO/IEC 2013 – All rights reserved

```
  FILE *file = fopen(filename, "r");
  if (file == NULL) {
    /* ... */
  }

  memset(&offset, 0, sizeof(offset));

  if (fsetpos(file, &offset) != 0) {  // diagnostic required
    /* ... */
  }

  return file;
}
```

5.42 Using an object overwritten by `getenv`, `localeconv`, `setlocale`, and `strerror` [libuse]

Rule

Using the object pointed to by the pointer returned by the `getenv`, `localeconv`, `setlocale`, and `strerror` functions after a subsequent call to the function shall be diagnosed.

Rationale

The object pointed to by the pointer returned by the `getenv`, `localeconv`, `setlocale`, and `strerror` functions may be overwritten by the subsequent call to the function.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the string returned by the first call to the C Standard Library function `getenv` is accessed, after the second call to `getenv`, in the call to the C Standard Library function `strcmp`.

```
int f(void) {
  char *tmpvar = getenv("TMP");
  char *tempvar = getenv("TEMP");

  if (!tmpvar || !tempvar) {
    /* ... */
  }

  return strcmp(tmpvar, tempvar) == 0;  // diagnostic required
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the string returned by the first call to the C Standard Library function `setlocale` is accessed, after the second call to `setlocale`, in the third call to `setlocale`.

```
void g(const char *name) {
  const char *save = setlocale(LC_ALL, 0);
  if (setlocale(LC_ALL, name)) {
    /* ... */
  }

  setlocale(LC_ALL, save);  // diagnostic required
}
```

EXAMPLE 3 In this noncompliant example, a diagnostic is required because the pointer returned from the first call to the C Standard Library function `strerror` is accessed, in the call to `fprintf`, after the second call to `strerror`.

```
void h(const char *a, const char *b) {
  errno = 0;
  unsigned long x = strtoul(a, NULL, 0);
  int e1 = ULONG_MAX == x ? errno : 0;

  errno = 0;
  unsigned long y = strtoul(b, NULL, 0);
  int e2 = ULONG_MAX == y ? errno : 0;
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
  char* err1 = strerror(e1);
  char* err2 = strerror(e2);
  fprintf(stderr, "parsing results: %s, %s", err1, err2); // diagnostic required
}
```

5.43 Using character values that are indistinguishable from EOF [chreof]

Rule

The following library character functions have return type int and return character values and the value EOF.

```
fgetc     getc     getchar
```

If the return value of one of the above library functions is stored into a variable of type char, any comparison of that stored value to a constant equal to the value of EOF shall be diagnosed.

Similarly, the following library wide character functions have return type wint_t and return wide character values and the value WEOF.

```
fgetwc    getwc    getwchar
```

If the return value of one of the above library functions is stored into a variable of type wchar_t, any comparison of that stored value to a constant equal to the value of WEOF shall be diagnosed.

Rationale

A character type cannot represent all character values plus the value of EOF, and a wide character type cannot represent all character values plus the value of WEOF.

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the result of the call to the C Standard Library function getchar is stored into a variable of type char, c, and c is compared to EOF.

```
void f(void) {
  char buf[BUFSIZ];
  char c;
  size_t i = 0;

  while ((c = getchar())
    != '\n' && c != EOF) {  // diagnostic required
    if (i < BUFSIZ - 1) {
      buf[i++] = c;
    }
  }

  buf[i] = '\0';
  printf("%s\n", buf);
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the result of the call to the C Standard Library function getwc is stored into a variable of type wchar_t, wc, and wc is compared to WEOF.

```
void g(void) {
  wchar_t buf[BUFSIZ];
  wchar_t wc;
  size_t i = 0;

  while ((wc = getwc(stdin))
    != '\n' && wc != WEOF) {  // diagnostic required
    if (i < BUFSIZ - 1) {
      buf[i++] = wc;
    }
```


© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.


```
  }

  buf[i] = '\0';
  wprintf("%s\n", buf);
}
```

5.44 Using identifiers that are reserved for the implementation [resident]

Rule

Declaring or defining an identifier in a context in which it is reserved or defines a reserved identifier as a macro name shall be diagnosed.

Rationale

According to ISO/IEC 9899:2011, 7.1.3, on reserved identifiers,

— All identifiers that begin with an underscore and either an uppercase letter or another underscore are always reserved for any use.

— All identifiers that begin with an underscore are always reserved for use as identifiers with file scope in both the ordinary and tag name spaces.

— Each macro name in any of the subclauses (including the future library directions) is reserved for use as specified if any one of its associated headers is included, unless explicitly stated otherwise.

— All identifiers with external linkage . . . (including the future library directions) and `errno` are always reserved for use as identifiers with external linkage.

— Each identifier with file scope listed in any of the above subclauses (including the future library directions) is reserved for use as a macro name and as an identifier with file scope in the same name space if any of its associated headers is included.

No other identifiers are reserved. The behavior of a program that declares or defines an identifier in a context in which it is reserved or defines a reserved identifier as a macro name is undefined. Trying to define a reserved identifier can result in its name conflicting with that used in implementation, which may or may not be detected at compile time.

C identifies the following undefined behavior:

UB	Description
106	The program declares or defines a reserved identifier, other than as allowed by 7.1.4 (7.1.3).

NOTE The POSIX® standard extends the set of identifiers reserved by C to include an open-ended set of its own.[4]

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the reserved identifier `errno` is redefined.

```
extern int errno;  // diagnostic required
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the identifier `_MY_HEADER_H_` defined in the header guard is reserved because it begins with an underscore and an uppercase letter.

```
#ifndef _MY_HEADER_H_
#define _MY_HEADER_H_  // diagnostic required

/* contents of <my_header.h> */

#endif /* _MY_HEADER_H_ */
```

EXAMPLE 3 In this compliant example, a diagnostic is not required because the identifier `MY_HEADER_H` defined in the header guard is not reserved.

© ISO/IEC 2013 – All rights reserved

```
#ifndef MY_HEADER_H
#define MY_HEADER_H

/* contents of <my_header.h> */

#endif /* MY_HEADER_H */
```

EXAMPLE 4 In this noncompliant example, a diagnostic is required because the file scope identifiers `_max_limit` and `_limit` are reserved because they begin with an underscore.

```
static const unsigned int _max_limit = 1024;  // diagnostic required
unsigned int _limit = 100;  // diagnostic required

unsigned int getValue(unsigned int count) {
  return count < _limit ? count : _limit;
}
```

EXAMPLE 5 In this compliant example, a diagnostic is not required because the file scope identifiers `max_limit` and `limit` are not reserved because they do not begin with an underscore.

```
static const unsigned int max_limit = 1024;
unsigned int limit = 100;

unsigned int getValue(unsigned int count){
  return count < limit ? count : limit;
}
```

EXAMPLE 6 In this noncompliant example, a diagnostic is required if `header <stdint.h>` is included because the identifier `SIZE_MAX` is reserved and the identifier `INTFAST16_LIMIT_MAX` is reserved because it begins with `INT` and ends with `_MAX`.

```
#define BUFSIZE 80

static const int_fast16_t INTFAST16_LIMIT_MAX = 12000;  // diagnostic required

void print_fast16(int_fast16_t val) {
  enum { SIZE_MAX = 80 };  // diagnostic required
  char buf[BUFSIZE];

  if (INTFAST16_LIMIT_MAX < val) {
    sprintf(buf, "The value is too large");
  } else {
    char fmt[BUFSIZE];
    snprintf(fmt, BUFSIZE, "%s %s", "The Value is %", PRIdFAST16);
    snprintf(buf, BUFSIZE, fmt, val);
  }
  fprintf(stdout, "\t%s\n", buf);
}
```

EXAMPLE 7 In this compliant example, a diagnostic is not required because the identifiers `BUFSIZE` and `MY_INTFAST16_UPPER_LIMIT` are not reserved.

```
static const int_fast16_t MY_INTFAST16_UPPER_LIMIT = 12000;

void print_fast16(int_fast16_t val) {
  enum { BUFSIZE = 80 };
  char buf[BUFSIZE];
  if (MY_INTFAST16_UPPER_LIMIT < val) {
    sprintf(buf, "The value is too large");
  } else {
    char fmt[BUFSIZE];
    snprintf(fmt, BUFSIZE, "%s %s", "The Value is %", PRIdFAST16);
    snprintf(buf, BUFSIZE, fmt, val);
  }
  fprintf(stdout, "\t%s\n", buf);
}
```

EXAMPLE 8 In this noncompliant example, a diagnostic is required because the identifiers for the C Standard Library functions `malloc` and `free` are reserved.

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

```
void *malloc(size_t nbytes) {  // diagnostic required
  void *ptr;
  /* ... */
  /* allocate storage from own pool and set ptr */
  return ptr;
}

void free(void *ptr) {  // diagnostic required
  /* ... */
  /* return storage to own pool */
}
```

EXAMPLE 9 In this compliant example, a diagnostic is not required because the reserved identifiers `malloc` and `free` are not used to define functions.

```
char *my_malloc(size_t nbytes) {
  if (nbytes > 0) {
    return (char *)malloc(nbytes);
  } else {
    return NULL;
  }
}

void my_free(char *p) {
  if(p != NULL) {
    free(p);
    p = NULL;
  }
}
```

5.45 Using invalid format strings [invfmtstr]

Rule

Supplying an unknown or invalid *conversion specification*; an invalid combination of *flag character*, *precision*, *length modifier*, *conversion specifier*; or a number and type of arguments to a formatted IO function that do not match the conversion specifiers in the format string shall be diagnosed.

Rationale

C identifies the following undefined behaviors:

UB	Description
155	In a call to one of the formatted output functions, a precision appears with a conversion specifier other than those described (7.21.6.1, 7.29.2.1).
156	A conversion specification for a formatted output function uses an asterisk to denote an argument-supplied field width or precision, but the corresponding argument is not provided (7.21.6.1, 7.29.2.1).
157	A conversion specification for a formatted output function uses a `#` or `0` flag with a conversion specifier other than those described (7.21.6.1, 7.29.2.1).
158	A conversion specification for one of the formatted input/output functions uses a length modifier with a conversion specifier other than those described (7.21.6.1, 7.21.6.2, 7.29.2.1, 7.29.2.2).
159	An `s` conversion specifier is encountered by one of the formatted output functions, and the argument is missing the null terminator (unless a precision is specified that does not require null termination) (7.21.6.1, 7.29.2.1).
160	An `n` conversion specification for one of the formatted input/output functions includes any flags, an assignment-suppressing character, a field width, or a precision (7.21.6.1, 7.21.6.2, 7.29.2.1, 7.29.2.2).
161	A `%` conversion specifier is encountered by one of the formatted input/output functions, but the complete conversion specification is not exactly `%%` (7.21.6.1, 7.21.6.2, 7.29.2.1, 7.29.2.2).
162	An invalid conversion specification is found in the format for one of the formatted input/output functions, or the `strftime` or `wcsftime` function (7.21.6.1, 7.21.6.2, 7.27.3.5, 7.29.2.1, 7.29.2.2, 7.29.5.1).

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Example(s)

EXAMPLE In this noncompliant example, a diagnostic is required because the arguments to `printf` do not match the conversion specifiers in the supplied format string.

```
void f(void) {
  const char *error_msg = "Resource not available to user.";
  int error_type = 3;
  /* ... */
  printf("Error (type %s): %d\n", error_type, error_msg);  // diagnostic required
}
```

5.46 Tainted, potentially mutilated, or out-of-domain integer values are used in a restricted sink [taintsink]

Rule

Values that are tainted (4.16), potentially mutilated, or out-of-domain integers and are used in an integer restricted sink shall be diagnosed.

Rationale

Using tainted (4.16), potentially mutilated, or out-of-domain integers in an integer restricted sink can result in accessing memory that is outside the bounds of existing objects.

Restricted sinks for integers include

— integer operands of any pointer arithmetic, including array indexing;

— the assignment expression for the declaration of a variable length array;

— the postfix expression preceding square brackets `[]` or the expression in square brackets `[]` of a subscripted designation of an element of an array object; and

— function arguments of type `size_t` or `rsize_t` (for example, an argument to a memory allocation function).

Example(s)

EXAMPLE 1 In this noncompliant example, a diagnostic is required because the tainted integer `size` is used to declare the size of the variable length array `vla`.

```
void f(const char *c_str) {
  size_t size;
  GET_TAINTED_INTEGER(size_t, size);
  char vla[size];  // diagnostic required

  strncpy(vla, c_str, size);
  vla[size - 1] = '\0';

  /* ... */
}
```

EXAMPLE 2 In this noncompliant example, a diagnostic is required because the tainted integer `color_index` is used in pointer arithmetic to index into the array `table`.

```
const char *table[] = { "black", "white", "blue", "green" };

const char *set_background_color(void) {
  int color_index;
  GET_TAINTED_INTEGER(int, color_index);

  const char *color = table[color_index];  // diagnostic required

  /* ... */
  return color;
}
```


© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Annex A
(informative)

Intra- to Interprocedural Transformations

Most of the examples given in the individual rules are simple and most often intraprocedural, only requiring analysis of a single function to either diagnose the violation or determine that there is no violation. Real-world code is often more complicated, requiring some form of interprocedural analysis. Collecting evidence for or against a violation may require examining the flow of data from one function to another, either explicitly through argument passing and return values or implicitly through global variables and the heap. There are many different approaches to interprocedural analysis, and this technical specification does not advocate one approach over another. This annex describes a transformational framework that can be used to generate more complex interprocedural examples from the simple intraprocedural examples given in the rules. These generated examples can then be used as an extended set of requirements for interprocedural analysis.

A.1 Function arguments and return values

The simplest case is a rule involving only one value, such as **Failing to detect and handle standard library errors** [liberr]. The following is an intraprocedural example:

```
FILE *fp = fopen(name, mode);
if (fp != NULL) /* checking for success */
  /* ... */
```

The basic interprocedural transformations are to pass the value into a function or return it from a function:

```
void check_it(FILE *fp) {
  if (fp != NULL) /* checking for success */
    /* ... */
}

/* ... */
check_it(fopen(name, mode));
```

or

```
/* a wrapper around fopen */
FILE *xfopen(const char *name, const char *mode) {
  return fopen(name, mode); /* return for checking elsewhere */
}

...
FILE *fp = xfopen(name, mode);
if (fp != NULL) /* checking for success */
   /* ... */
```

An important special case combines argument passing and returning to form an identity operation:

```
/* trivial example */
FILE *identity(FILE *fp) {
  return fp;
}

FILE *fp = identity(fopen(name, mode));
if (fp != NULL) /* checking for success */
  /* ... */
```

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

A.2 Indirection

An additional transformation is indirection:

```
void check_indirect(FILE **pfp) {
  if (*pfp != NULL) /* checking for success */
    /* ... */
}

/* ... */
FILE *fp = fopen(name, mode);
check_indirect(&fp);

void return_result_thru_param(const char *name, const char *mode,
                              FILE **result) {
  *result = fopen(name, mode);
}

/* ... */
FILE *fp;
return_result_thru_param(name, mode, &fp);
if (fp != NULL) /* checking for success */
  /* ... */
```

Indirection can also involve fields of structs or unions. Indirection can be applied recursively, although precise handling of indirection often becomes increasingly expensive as the number of levels of indirection increases.

When a rule involves multiple values, such as **Forming or using out-of-bounds pointers or array subscripts** [invptr] (where a violation is an interaction between an array and an index), these transformations apply separately or in combination to each of the values. The following is a simple intraprocedural example:

```
int array[2];
int index = 2; /* part of violation */
array[index] = 0; /* violation */
```

Applying some of the interprocedural transformations yields

```
void indexer(int *array, int index) {
  array[index] = 0; /* part of violation */
}

/* ... */
int array[2];
int index = 2; /* part of violation */
indexer(array, index); /* violation */
```

or

```
static int array[2];
int *get_array() {
  return array; /* part of violation */
}

/* ... */
get_array()[2] = 0; /* violation */
```

or

```
struct array_params {
  int *array;
  int index;
};

void indexer(struct array_params *ap) {
  ap->array[ap->index] = 0; /* part of violation */
}
```

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.


```
/* ... */
int array[2];
struct array_params params;
params.array = array; /* part of violation */
params.index = 2; /* part of violation */
indexer(&params); /* violation */
```

These violations involve three steps: the array, the index, and the address arithmetic, where each could occur in a different function:

```
int *add(int *base, int offset) {
  return base + offset; /* part of violation */
}

/* ... */
int array[2];
int index = 2; /* part of violation */
*add(array, index) = 0; /* violation */
```

Indirection may also be applied to a pointer being returned from a function:

```
const int *ptr_to_index() {
  static int rv;
  rv = 2; /* part of violation */
  return &rv; /* part of violation */
}
/* ... */
int array[2];
array[*ptr_to_index()] = 0; /* violation */
```

A.3 Transformation involving standard library functions

The following transformation involves tracing the flow of data through the C Standard Library function `strchr()` that returns a pointer to an element in the array specified by its first argument if the element's value equals that of the second argument and a null pointer otherwise. Because the effects and the return value of the function are precisely specified, an analyzer can determine that the assignment to the `*slash` object modifies an element of the `const` string `pathname`, potentially causing undefined behavior.

```
const char* basename(const char *pathname) {
  char *slash;

  slash = strchr(pathname, '/');
  if (slash) {
    *slash++ = '\0';   /* violates EXP40-C. Do not modify constant values */
    return slash;
  }

  return pathname;
}
```

Note that interprocedural analysis could identify such data flow through an arbitrary function, but in the case of standard library functions, it is not necessary to analyze the function's implementation to derive the flow.

A.4 Data flow through globals

Interprocedural data flow may not be explicit through argument passing and value returning; it may involve `extern` or `static` variables:

```
int global_index;

void set_it() {
  global_index = 2; /* part of violation */
}

/* ... */
```

© ISO/IEC 2013 – All rights reserved

```
int array[2];
set_it(); /* part of violation */
array[global_index] = 0; /* violation */
```

A.5 Data flow through the heap

Data may flow from one function to another through heap locations:

```
struct some_record {
  int index;
  /* ... */
};

/* returns the heap-allocated record (created elsewhere)
 * matching 'key'
 */
struct some_record *find_record(const char *key);

void set_it() {
  struct some_record *record = find_record("xyz");
  record->index = 2;
}

/* ... */
int array[2];
set_it(); /* part of violation */
struct some_record *record = find_record("xyz"); /* part of violation */
array[record->index] = 0; /* violation */

/* note that find_record() could even be called before set_it() */
```

A.6 Combined example

This example combines some of these transformations in a way that a reasonably sophisticated interprocedural analysis might diagnose:

```
struct trouble {
  int *array;
  int index;
  int *effective_address;
};

void set_array(struct trouble *t, int *array) {
  t->array = array; /* part of violation */
}

void set_index(struct trouble *t, int *index) {
  t->index = *index; /* part of violation */
}

void compute_effective_address(struct trouble *t) {
  t->effective_address = t->array + t->index; /* part of violation */
}

void store(struct trouble *t, int value) {
  *t->effective_address = value; /* part of violation */
}

...
int array[2];
int index = 2; /* part of violation */
struct trouble t;
set_array(t, array); /* part of violation */
set_index(t, &index); /* part of violation */
compute_effective_address(&t); /* violation */
store(&t, 0); /* violation */
```


© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Annex B
(informative)

Undefined Behavior

According to C (as summarized in ISO/IEC 9899:2011, J.2), the behavior of a program is undefined in the circumstances outlined in Table B.1. The parenthesized section numbers refer to the section of ISO/IEC 9899:2011 that identifies the undefined behavior.

Table B.1 — Undefined behaviors

UB	Description
1	A "shall" or "shall not" requirement that appears outside of a constraint is violated (clause 4).
2	A nonempty source file does not end in a new-line character which is not immediately preceded by a backslash character or ends in a partial preprocessing token or comment (5.1.1.2).
3	Token concatenation produces a character sequence matching the syntax of a universal character name (5.1.1.2).
4	A program in a hosted environment does not define a function named `main` using one of the specified forms (5.1.2.2.1).
5	The execution of a program contains a data race (5.1.2.4).
6	A character not in the basic source character set is encountered in a source file, except in an identifier, a character constant, a string literal, a header name, a comment, or a preprocessing token that is never converted to a token (5.2.1).
7	An identifier, comment, string literal, character constant, or header name contains an invalid multibyte character or does not begin and end in the initial shift state (5.2.1.2).
8	The same identifier has both internal and external linkage in the same translation unit (6.2.2).
9	An object is referred to outside of its lifetime (6.2.4).
10	The value of a pointer to an object whose lifetime has ended is used (6.2.4).
11	The value of an object with automatic storage duration is used while it is indeterminate (6.2.4, 6.7.9, 6.8).
12	A trap representation is read by an lvalue expression that does not have character type (6.2.6.1).
13	A trap representation is produced by a side effect that modifies any part of the object using an lvalue expression that does not have character type (6.2.6.1).
14	The operands to certain operators are such that they could produce a negative zero result, but the implementation does not support negative zeros (6.2.6.2).
15	Two declarations of the same object or function specify types that are not compatible (6.2.7).
16	A program requires the formation of a composite type from a variable length array type whose size is specified by an expression that is not evaluated (6.2.7).
17	Conversion to or from an integer type produces a value outside the range that can be represented (6.3.1.4).
18	Demotion of one real floating type to another produces a value outside the range that can be represented (6.3.1.5).
19	An lvalue does not designate an object when evaluated (6.3.2.1).
20	A non-array lvalue with an incomplete type is used in a context that requires the value of the designated object (6.3.2.1).
21	An lvalue designating an object of automatic storage duration that could have been declared with the `register` storage class is used in a context that requires the value of the designated object, but the object is uninitialized. (6.3.2.1).
22	An lvalue having array type is converted to a pointer to the initial element of the array, and the array object has register storage class (6.3.2.1).
23	An attempt is made to use the value of a void expression, or an implicit or explicit conversion (except to `void`) is applied to a void expression (6.3.2.2).

© ISO/IEC 2013 – All rights reserved

Table B.1 *(continued)*

UB	Description
24	Conversion of a pointer to an integer type produces a value outside the range that can be represented (6.3.2.3).
25	Conversion between two pointer types produces a result that is incorrectly aligned (6.3.2.3).
26	A pointer is used to call a function whose type is not compatible with the referenced type (6.3.2.3).
27	An unmatched `'` or `"` character is encountered on a logical source line during tokenization (6.4).
28	A reserved keyword token is used in translation phase 7 or 8 for some purpose other than as a keyword (6.4.1).
29	A universal character name in an identifier does not designate a character whose encoding falls into one of the specified ranges (6.4.2.1).
30	The initial character of an identifier is a universal character name designating a digit (6.4.2.1).
31	Two identifiers differ only in nonsignificant characters (6.4.2.1).
32	The identifier `__func__` is explicitly declared (6.4.2.2).
33	The program attempts to modify a string literal (6.4.5).
34	The characters `'`, `\`, `"`, `//`, or `/*` occur in the sequence between the < and > delimiters, or the characters `'`, `\`, `//`, or `/*` occur in the sequence between the `"` delimiters, in a header name preprocessing token (6.4.7).
35	A side effect on a scalar object is unsequenced relative to either a different side effect on the same scalar object or a value computation using the value of the same scalar object (6.5).
36	An exceptional condition occurs during the evaluation of an expression (6.5).
37	An object has its stored value accessed other than by an lvalue of an allowable type (6.5).
38	For a call to a function without a function prototype in scope, the number of arguments does not equal the number of parameters (6.5.2.2).
39	For call to a function without a function prototype in scope where the function is defined with a function prototype, either the prototype ends with an ellipsis or the types of the arguments after promotion are not compatible with the types of the parameters (6.5.2.2).
40	For a call to a function without a function prototype in scope where the function is not defined with a function prototype, the types of the arguments after promotion are not compatible with those of the parameters after promotion (with certain exceptions) (6.5.2.2).
41	A function is defined with a type that is not compatible with the type (of the expression) pointed to by the expression that denotes the called function (6.5.2.2).
42	A member of an atomic structure or union is accessed (6.5.2.3).
43	The operand of the unary `*` operator has an invalid value (6.5.3.2).
44	A pointer is converted to other than an integer or pointer type (6.5.4).
45	The value of the second operand of the `/` or `%` operator is zero (6.5.5).
46	Addition or subtraction of a pointer into, or just beyond, an array object and an integer type produces a result that does not point into, or just beyond, the same array object (6.5.6).
47	Addition or subtraction of a pointer into, or just beyond, an array object and an integer type produces a result that points just beyond the array object and is used as the operand of a unary `*` operator that is evaluated (6.5.6).
48	Pointers that do not point into, or just beyond, the same array object are subtracted (6.5.6).
49	An array subscript is out of range, even if an object is apparently accessible with the given subscript (as in the lvalue expression `a[1][7]` given the declaration `int a[4][5]`) (6.5.6).
50	The result of subtracting two pointers is not representable in an object of type `ptrdiff_t` (6.5.6).
51	An expression is shifted by a negative number or by an amount greater than or equal to the width of the promoted expression (6.5.7).
52	An expression having signed promoted type is left-shifted and either the value of the expression is negative or the result of shifting would be not be representable in the promoted type (6.5.7).
53	Pointers that do not point to the same aggregate or union (nor just beyond the same array object) are compared using relational operators (6.5.8).
54	An object is assigned to an inexactly overlapping object or to an exactly overlapping object with incompatible type (6.5.16.1).

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Table B.1 *(continued)*

UB	Description
55	An expression that is required to be an integer constant expression does not have an integer type; has operands that are not integer constants, enumeration constants, character constants, `sizeof` expressions whose results are integer constants, `_Alignof` expressions, or immediately-cast floating constants; or contains casts (outside operands to `sizeof` and `_Alignof` operators) other than conversions of arithmetic types to integer types (6.6).
56	A constant expression in an initializer is not, or does not evaluate to, one of the following: an arithmetic constant expression, a null pointer constant, an address constant, or an address constant for a complete object type plus or minus an integer constant expression (6.6).
57	An arithmetic constant expression does not have arithmetic type; has operands that are not integer constants, floating constants, enumeration constants, character constants, `sizeof` expressions whose results are integer constants, or `_Alignof` expressions; or contains casts (outside operands to `sizeof` or `_Alignof` operators) other than conversions of arithmetic types to arithmetic types (6.6).
58	The value of an object is accessed by an array-subscript `[]`, member-access `.` or `->`, address `&`, or indirection `*` operator or a pointer cast in creating an address constant (6.6).
59	An identifier for an object is declared with no linkage and the type of the object is incomplete after its declarator, or after its init-declarator if it has an initializer (6.7).
60	A function is declared at block scope with an explicit storage-class specifier other than `extern` (6.7.1).
61	A structure or union is defined without any named members (including those specified indirectly via anonymous structures and unions) (6.7.2.1).
62	An attempt is made to access, or generate a pointer to just past, a flexible array member of a structure when the referenced object provides no elements for that array (6.7.2.1).
63	When the complete type is needed, an incomplete structure or union type is not completed in the same scope by another declaration of the tag that defines the content (6.7.2.3).
64	An attempt is made to modify an object defined with a const-qualified type through use of an lvalue with non-const-qualified type (6.7.3).
65	An attempt is made to refer to an object defined with a volatile-qualified type through use of an lvalue with non-volatile-qualified type (6.7.3).
66	The specification of a function type includes any type qualifiers (6.7.3).
67	Two qualified types that are required to be compatible do not have the identically qualified version of a compatible type (6.7.3).
68	An object which has been modified is accessed through a restrict-qualified pointer to a const-qualified type, or through a restrict-qualified pointer and another pointer that are not both based on the same object (6.7.3.1).
69	A restrict-qualified pointer is assigned a value based on another restricted pointer whose associated block neither began execution before the block associated with this pointer, nor ended before the assignment (6.7.3.1).
70	A function with external linkage is declared with an `inline` function specifier, but is not also defined in the same translation unit (6.7.4).
71	A function declared with a `_Noreturn` function specifier returns to its caller (6.7.4).
72	The definition of an object has an alignment specifier and another declaration of that object has a different alignment specifier (6.7.5).
73	Declarations of an object in different translation units have different alignment specifiers (6.7.5).
74	Two pointer types that are required to be compatible are not identically qualified, or are not pointers to compatible types (6.7.6.1).
75	The size expression in an array declaration is not a constant expression and evaluates at program execution time to a nonpositive value (6.7.6.2).
76	In a context requiring two array types to be compatible, they do not have compatible element types, or their size specifiers evaluate to unequal values (6.7.6.2).
77	A declaration of an array parameter includes the keyword `static` within the `[` and `]` and the corresponding argument does not provide access to the first element of an array with at least the specified number of elements (6.7.6.3).
78	A storage-class specifier or type qualifier modifies the keyword `void` as a function parameter type list (6.7.6.3).
79	In a context requiring two function types to be compatible, they do not have compatible return types, or their parameters disagree in use of the ellipsis terminator or the number and type of parameters (after default argument promotion, when there is no parameter type list or when one type is specified by a function definition with an identifier list) (6.7.6.3).

© ISO/IEC 2013 – All rights reserved

Table B.1 *(continued)*

UB	Description
80	The value of an unnamed member of a structure or union is used (6.7.9).
81	The initializer for a scalar is neither a single expression nor a single expression enclosed in braces (6.7.9).
82	The initializer for a structure or union object that has automatic storage duration is neither an initializer list nor a single expression that has compatible structure or union type (6.7.9).
83	The initializer for an aggregate or union, other than an array initialized by a string literal, is not a brace-enclosed list of initializers for its elements or members (6.7.9).
84	An identifier with external linkage is used, but in the program there does not exist exactly one external definition for the identifier, or the identifier is not used and there exist multiple external definitions for the identifier (6.9).
85	A function definition includes an identifier list, but the types of the parameters are not declared in a following declaration list (6.9.1).
86	An adjusted parameter type in a function definition is not a complete object type (6.9.1).
87	A function that accepts a variable number of arguments is defined without a parameter type list that ends with the ellipsis notation (6.9.1).
88	The `}` that terminates a function is reached, and the value of the function call is used by the caller (6.9.1).
89	An identifier for an object with internal linkage and an incomplete type is declared with a tentative definition (6.9.2).
90	The token `defined` is generated during the expansion of a `#if` or `#elif` preprocessing directive, or the use of the `defined` unary operator does not match one of the two specified forms prior to macro replacement (6.10.1).
91	The `#include` preprocessing directive that results after expansion does not match one of the two header name forms (6.10.2).
92	The character sequence in an `#include` preprocessing directive does not start with a letter (6.10.2).
93	There are sequences of preprocessing tokens within the list of macro arguments that would otherwise act as preprocessing directives (6.10.3).
94	The result of the preprocessing operator `#` is not a valid character string literal (6.10.3.2).
95	The result of the preprocessing operator `##` is not a valid preprocessing token (6.10.3.3).
96	The `#line` preprocessing directive that results after expansion does not match one of the two well-defined forms, or its digit sequence specifies zero or a number greater than 2147483647 (6.10.4).
97	A non-`STDC #pragma` preprocessing directive that is documented as causing translation failure or some other form of undefined behavior is encountered (6.10.6).
98	A `#pragma STDC` preprocessing directive does not match one of the well-defined forms (6.10.6).
99	The name of a predefined macro, or the identifier `defined`, is the subject of a `#define` or `#undef` preprocessing directive (6.10.8).
100	An attempt is made to copy an object to an overlapping object by use of a library function, other than as explicitly allowed (e.g., `memmove`) (clause 7).
101	A file with the same name as one of the standard headers, not provided as part of the implementation, is placed in any of the standard places that are searched for included source files (7.1.2).
102	A header is included within an external declaration or definition (7.1.2).
103	A function, object, type, or macro that is specified as being declared or defined by some standard header is used before any header that declares or defines it is included (7.1.2).
104	A standard header is included while a macro is defined with the same name as a keyword (7.1.2).
105	The program attempts to declare a library function itself, rather than via a standard header, but the declaration does not have external linkage (7.1.2).
106	The program declares or defines a reserved identifier, other than as allowed by 7.1.4 (7.1.3).
107	The program removes the definition of a macro whose name begins with an underscore and either an uppercase letter or another underscore (7.1.3).
108	An argument to a library function has an invalid value or a type not expected by a function with variable number of arguments (7.1.4).
109	The pointer passed to a library function array parameter does not have a value such that all address computations and object accesses are valid (7.1.4).
110	The macro definition of `assert` is suppressed in order to access an actual function (7.2).

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Table B.1 *(continued)*

UB	Description
111	The argument to the `assert` macro does not have a scalar type (7.2).
112	The `CX_LIMITED_RANGE`, `FENV_ACCESS`, or `FP_CONTRACT` pragma is used in any context other than outside all external declarations or preceding all explicit declarations and statements inside a compound statement (7.3.4, 7.6.1, 7.12.2).
113	The value of an argument to a character handling function is neither equal to the value of `EOF` nor representable as an `unsigned char` (7.4).
114	A macro definition of `errno` is suppressed in order to access an actual object, or the program defines an identifier with the name `errno` (7.5).
115	Part of the program tests floating-point status flags, sets floating-point control modes, or runs under non-default mode settings, but was translated with the state for the `FENV_ACCESS` pragma "off" (7.6.1).
116	The exception-mask argument for one of the functions that provide access to the floating-point status flags has a nonzero value not obtained by bitwise OR of the floating-point exception macros (7.6.2).
117	The `fesetexceptflag` function is used to set floating-point status flags that were not specified in the call to the `fegetexceptflag` function that provided the value of the corresponding `fexcept_t` object (7.6.2.4).
118	The argument to `fesetenv` or `feupdateenv` is neither an object set by a call to `fegetenv` or `feholdexcept`, nor is it an environment macro (7.6.4.3, 7.6.4.4).
119	The value of the result of an integer arithmetic or conversion function cannot be represented (7.8.2.1, 7.8.2.2, 7.8.2.3, 7.8.2.4, 7.22.6.1, 7.22.6.2, 7.22.1).
120	The program modifies the string pointed to by the value returned by the `setlocale` function (7.11.1.1).
121	The program modifies the structure pointed to by the value returned by the `localeconv` function (7.11.2.1).
122	A macro definition of `math_errhandling` is suppressed or the program defines an identifier with the name `math_errhandling` (7.12).
123	An argument to a floating-point classification or comparison macro is not of real floating type (7.12.3, 7.12.14).
124	A macro definition of `setjmp` is suppressed in order to access an actual function, or the program defines an external identifier with the name `setjmp` (7.13).
125	An invocation of the `setjmp` macro occurs other than in an allowed context (7.13.2.1).
126	The `longjmp` function is invoked to restore a nonexistent environment (7.13.2.1).
127	After a `longjmp`, there is an attempt to access the value of an object of automatic storage duration that does not have volatile-qualified type, local to the function containing the invocation of the corresponding `setjmp` macro, that was changed between the `setjmp` invocation and `longjmp` call (7.13.2.1).
128	The program specifies an invalid pointer to a signal handler function (7.14.1.1).
129	A signal handler returns when the signal corresponded to a computational exception (7.14.1.1).
130	A signal handler called in response to `SIGFPE`, `SIGILL`, `SIGSEGV`, or any other implementation-defined value corresponding to a computational exception returns (7.14.1.1).
131	A signal occurs as the result of calling the `abort` or `raise` function, and the signal handler calls the `raise` function (7.14.1.1).
132	A signal occurs other than as the result of calling the `abort` or `raise` function, and the signal handler refers to an object with static or thread storage duration that is not a lock-free atomic object other than by assigning a value to an object declared as `volatile sig_atomic_t`, or calls any function in the standard library other than the `abort` function, the `_Exit` function, the `quick_exit` function, or the `signal` function (for the same signal number) (7.14.1.1).
133	The value of `errno` is referred to after a signal occurred other than as the result of calling the `abort` or `raise` function and the corresponding signal handler obtained a `SIG_ERR` return from a call to the `signal` function (7.14.1.1).
134	A signal is generated by an asynchronous signal handler (7.14.1.1).
135	The `signal` function is used in a multi-threaded program (7.14.1.1).
136	A function with a variable number of arguments attempts to access its varying arguments other than through a properly declared and initialized `va_list` object, or before the `va_start` macro is invoked (7.16, 7.16.1.1, 7.16.1.4).
137	The macro `va_arg` is invoked using the parameter `ap` that was passed to a function that invoked the macro `va_arg` with the same parameter (7.16).

© ISO/IEC 2013 – All rights reserved

Table B.1 *(continued)*

UB	Description
138	A macro definition of `va_start`, `va_arg`, `va_copy`, or `va_end` is suppressed in order to access an actual function, or the program defines an external identifier with the name `va_copy` or `va_end` (7.16.1).
139	The `va_start` or `va_copy` macro is invoked without a corresponding invocation of the `va_end` macro in the same function, or vice versa (7.16.1, 7.16.1.2, 7.16.1.3, 7.16.1.4).
140	The type parameter to the `va_arg` macro is not such that a pointer to an object of that type can be obtained simply by postfixing a `*` (7.16.1.1).
141	The `va_arg` macro is invoked when there is no actual next argument, or with a specified type that is not compatible with the promoted type of the actual next argument, with certain exceptions (7.16.1.1).
142	The `va_copy` or `va_start` macro is called to initialize a `va_list` that was previously initialized by either macro without an intervening invocation of the `va_end` macro for the same `va_list` (7.16.1.2, 7.16.1.4).
143	The parameter *parmN* of a `va_start` macro is declared with the `register` storage class, with a function or array type, or with a type that is not compatible with the type that results after application of the default argument promotions (7.16.1.4).
144	The member designator parameter of an `offsetof` macro is an invalid right operand of the `.` operator for the *type* parameter, or designates a bit-field (7.19).
145	The argument in an instance of one of the integer-constant macros is not a decimal, octal, or hexadecimal constant, or it has a value that exceeds the limits for the corresponding type (7.20.4).
146	A byte input/output function is applied to a wide-oriented stream, or a wide character input/output function is applied to a byte-oriented stream (7.21.2).
147	Use is made of any portion of a file beyond the most recent wide character written to a wide-oriented stream (7.21.2).
148	The value of a pointer to a `FILE` object is used after the associated file is closed (7.21.3).
149	The stream for the `fflush` function points to an input stream or to an update stream in which the most recent operation was input (7.21.5.2).
150	The string pointed to by the `mode` argument in a call to the `fopen` function does not exactly match one of the specified character sequences (7.21.5.3).
151	An output operation on an update stream is followed by an input operation without an intervening call to the `fflush` function or a file positioning function, or an input operation on an update stream is followed by an output operation with an intervening call to a file positioning function (7.21.5.3).
152	An attempt is made to use the contents of the array that was supplied in a call to the `setvbuf` function (7.21.5.6).
153	There are insufficient arguments for the format in a call to one of the formatted input/output functions, or an argument does not have an appropriate type (7.21.6.1, 7.21.6.2, 7.29.2.1, 7.29.2.2).
154	The format in a call to one of the formatted input/output functions or to the `strftime` or `wcsftime` function is not a valid multibyte character sequence that begins and ends in its initial shift state (7.21.6.1, 7.21.6.2, 7.27.3.5, 7.29.2.1, 7.29.2.2, 7.29.5.1).
155	In a call to one of the formatted output functions, a precision appears with a conversion specifier other than those described (7.21.6.1, 7.29.2.1).
156	A conversion specification for a formatted output function uses an asterisk to denote an argument-supplied field width or precision, but the corresponding argument is not provided (7.21.6.1, 7.29.2.1).
157	A conversion specification for a formatted output function uses a `#` or `0` flag with a conversion specifier other than those described (7.21.6.1, 7.29.2.1).
158	A conversion specification for one of the formatted input/output functions uses a length modifier with a conversion specifier other than those described (7.21.6.1, 7.21.6.2, 7.29.2.1, 7.29.2.2).
159	An `s` conversion specifier is encountered by one of the formatted output functions, and the argument is missing the null terminator (unless a precision is specified that does not require null termination) (7.21.6.1, 7.29.2.1).
160	An `n` conversion specification for one of the formatted input/output functions includes any flags, an assignment-suppressing character, a field width, or a precision (7.21.6.1, 7.21.6.2, 7.29.2.1, 7.29.2.2).
161	A `%` conversion specifier is encountered by one of the formatted input/output functions, but the complete conversion specification is not exactly `%%` (7.21.6.1, 7.21.6.2, 7.29.2.1, 7.29.2.2).
162	An invalid conversion specification is found in the format for one of the formatted input/output functions, or the `strftime` or `wcsftime` function (7.21.6.1, 7.21.6.2, 7.27.3.5, 7.29.2.1, 7.29.2.2, 7.29.5.1).
163	The number of characters or wide characters transmitted by a formatted output function (or written to an array, or that would have been written to an array) is greater than `INT_MAX` (7.21.6.1, 7.29.2.1).

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Table B.1 *(continued)*

UB	Description
164	The number of input items assigned by a formatted input function is greater than `INT_MAX` (7.21.6.2, 7.29.2.2).
165	The result of a conversion by one of the formatted input functions cannot be represented in the corresponding object, or the receiving object does not have an appropriate type (7.21.6.2, 7.29.2.2).
166	A `c`, `s`, or `[` conversion specifier is encountered by one of the formatted input functions, and the array pointed to by the corresponding argument is not large enough to accept the input sequence (and a null terminator if the conversion specifier is `s` or `[`) (7.21.6.2, 7.29.2.2).
167	A `c`, `s`, or `[` conversion specifier with an `l` qualifier is encountered by one of the formatted input functions, but the input is not a valid multibyte character sequence that begins in the initial shift state (7.21.6.2, 7.29.2.2).
168	The input item for a `%p` conversion by one of the formatted input functions is not a value converted earlier during the same program execution (7.21.6.2, 7.29.2.2).
169	The `vfprintf`, `vfscanf`, `vprintf`, `vscanf`, `vsnprintf`, `vsprintf`, `vsscanf`, `vfwprintf`, `vfwscanf`, `vswprintf`, `vswscanf`, `vwprintf`, or `vwscanf` function is called with an improperly initialized `va_list` argument, or the argument is used (other than in an invocation of `va_end`) after the function returns (7.21.6.8, 7.21.6.9, 7.21.6.10, 7.21.6.11, 7.21.6.12, 7.21.6.13, 7.21.6.14, 7.29.2.5, 7.29.2.6, 7.29.2.7, 7.29.2.8, 7.29.2.9, 7.29.2.10).
170	The contents of the array supplied in a call to the `fgets` or `fgetws` function are used after a read error occurred (7.21.7.2, 7.29.3.2).
171	The file position indicator for a binary stream is used after a call to the `ungetc` function where its value was zero before the call (7.21.7.10).
172	The file position indicator for a stream is used after an error occurred during a call to the `fread` or `fwrite` function (7.21.8.1, 7.21.8.2).
173	A partial element read by a call to the `fread` function is used (7.21.8.1).
174	The `fseek` function is called for a text stream with a nonzero offset and either the offset was not returned by a previous successful call to the `ftell` function on a stream associated with the same file or `whence` is not `SEEK_SET` (7.21.9.2).
175	The `fsetpos` function is called to set a position that was not returned by a previous successful call to the `fgetpos` function on a stream associated with the same file (7.21.9.3).
176	A non-null pointer returned by a call to the `calloc`, `malloc`, or `realloc` function with a zero requested size is used to access an object (7.22.3).
177	The value of a pointer that refers to space deallocated by a call to the `free` or `realloc` function is used (7.22.3).
178	The alignment requested of the `aligned_alloc` function is not valid or not supported by the implementation, or the size requested is not an integral multiple of the alignment (7.22.3.1).
179	The pointer argument to the `free` or `realloc` function does not match a pointer earlier returned by a memory management function, or the space has been deallocated by a call to `free` or `realloc` (7.22.3.3, 7.22.3.5).
180	The value of the object allocated by the `malloc` function is used (7.22.3.4).
181	The value of any bytes in a new object allocated by the `realloc` function beyond the size of the old object are used (7.22.3.5).
182	The program calls the `exit` or `quick_exit` function more than once, or calls both functions (7.22.4.4, 7.22.4.7).
183	During the call to a function registered with the `atexit` or `at_quick_exit` function, a call is made to the `longjmp` function that would terminate the call to the registered function (7.22.4.4, 7.22.4.7).
184	The string set up by the `getenv` or `strerror` function is modified by the program (7.22.4.6, 7.24.6.2).
185	A signal is raised while the `quick_exit` function is executing (7.22.4.7).
186	A command is executed through the `system` function in a way that is documented as causing termination or some other form of undefined behavior (7.22.4.8).
187	A searching or sorting utility function is called with an invalid pointer argument, even if the number of elements is zero (7.22.5).
188	The comparison function called by a searching or sorting utility function alters the contents of the array being searched or sorted, or returns ordering values inconsistently (7.22.5).
189	The array being searched by the `bsearch` function does not have its elements in proper order (7.22.5.1).
190	The current conversion state is used by a multibyte/wide character conversion function after changing the `LC_CTYPE` category (7.22.7).
191	A string or wide string utility function is instructed to access an array beyond the end of an object (7.24.1, 7.29.4).

© ISO/IEC 2013 – All rights reserved

Table B.1 *(continued)*

UB	Description
192	A string or wide string utility function is called with an invalid pointer argument, even if the length is zero (7.24.1, 7.29.4).
193	The contents of the destination array are used after a call to the `strxfrm`, `strftime`, `wcsxfrm`, or `wcsftime` function in which the specified length was too small to hold the entire null-terminated result (7.24.4.5, 7.27.3.5, 7.29.4.4.4, 7.29.5.1).
194	The first argument in the very first call to the `strtok` or `wcstok` is a null pointer (7.24.5.8, 7.29.4.5.7).
195	The type of an argument to a type-generic macro is not compatible with the type of the corresponding parameter of the selected function (7.25).
196	A complex argument is supplied for a generic parameter of a type-generic macro that has no corresponding complex function (7.25).
197	At least one member of the broken-down time passed to `asctime` contains a value outside its normal range, or the calculated year exceeds four digits or is less than the year 1000 (7.27.3.1).
198	The argument corresponding to an s specifier without an `l` qualifier in a call to the `fwprintf` function does not point to a valid multibyte character sequence that begins in the initial shift state (7.29.2.11).
199	In a call to the `wcstok` function, the object pointed to by `ptr` does not have the value stored by the previous call for the same wide string (7.29.4.5.7).
200	An `mbstate_t` object is used inappropriately (7.29.6).
201	The value of an argument of type `wint_t` to a wide character classification or case mapping function is neither equal to the value of `WEOF` nor representable as a `wchar_t` (7.30.1).
202	The `iswctype` function is called using a different `LC_CTYPE` category from the one in effect for the call to the `wctype` function that returned the description (7.30.2.2.1).
203	The `towctrans` function is called using a different `LC_CTYPE` category from the one in effect for the call to the `wctrans` function that returned the description (7.30.3.2.1).

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Annex C
(informative)

Related Guidelines and References

Rule	Related Guidelines	References
5.1 Accessing an object through a pointer to an incompatible type [ptrcomp]	CERT C Secure Coding Standard: EXP39-C. Do not access a variable through a pointer of an incompatible type[5]	n/a
5.2 Accessing freed memory [accfree]	CERT C Secure Coding Standard, MEM30-C. Do not access freed memory[5] ISO/IEC TR 24772, "Dangling References to Stack Frames [DCM]" and "Dangling Reference to Heap [XYK]"[6] MISRA-C 2004, Rule 17.6[7] MITRE CWE, CWE-416: Use after Free[8]	[3] Chapter 4, "Dynamic Memory Management" [9] Section 7.8.5, "Storage management" [10] Freed Memory [11] Section 5.2.19, "Using freed memory"
5.3 Accessing shared objects in signal handlers [accsig]	CERT C Secure Coding Standard, SIG31-C. Do not access or modify shared objects in signal handlers[5] Rationale for International Standard – C, "Signals and interrupts"[12] MITRE CWE, CWE-662: Improper Synchronization[8]	[13] Chapter 13, "Synchronization and State" [14] longjmp [15] signal Man Page [16]
5.4 No assignment in conditional expressions [boolasgn]	CERT C Secure Coding Standard, EXP18-C. Do not perform assignments in selection statements[5] ISO/IEC TR 24772, "Likely Incorrect Expressions [KOA]"[6] MITRE CWE: CWE-480: Use of Incorrect Operator CWE-481: Assigning instead of Comparing[8]	[17] Section 2.7.2, "Errors of omission and addition"
5.5 Calling functions in the C Standard Library other than `abort`, `_Exit`, and `signal` from within a signal handler [asyncsig]	CERT C Secure Coding Standard: SIG30-C. Call only asynchronous-safe functions within signal handlers SIG33-C. Do not recursively invoke the raise() function[5] Rationale for International Standard – C, Section 5.2.3, "Signals and interrupts"[12] MITRE CWE, CWE-479: Signal Handler Use of a Non-reentrant Function[8]	[13] Chapter 13, "Synchronization and State" [14] longjmp [15] signal Manual Page [16] "Delivering Signals for Fun and Profit"
5.6 Calling functions with incorrect arguments [argcomp]	CERT C Secure Coding Standard, DCL35-C. Call functions with the correct number and type of arguments[5] ISO/IEC TR 24772, "Subprogram Signature Mismatch [OTR]"[6] MISRA-C 2004, Rule 16.6[7] MITRE CWE, CWE-628: Function Call with Incorrectly Specified Arguments[8]	[18] CVE-2006-1174 [19] Section 2.6.1, "Incorrect routine or arguments"

© ISO/IEC 2013 – All rights reserved

Rule	Related Guidelines	References
5.7 Calling `signal` from interruptible signal handlers [sigcall]	CERT C Secure Coding Standard, SIG34-C. Do not call signal() from within interruptible signal handlers[5] MITRE CWE, CWE-479: Signal Handler Use of a Non-reentrant Function[8]	n/a
5.8 Calling `system` [syscall]	CERT C Secure Coding Standard, ENV04-C. Do not call system() if you do not need a command processor[5] ISO/IEC TR 24772, "Unquoted Search Path or Element [XZQ]"[6] MITRE CWE: CWE-78: Improper Neutralization of Special Elements Used in an OS Command ("OS Command Injection") CWE-88: Argument Injection or Modification[8]	[14] environ, execl, execv, execle, execve, execlp, execvp—execute a file, popen, unlink, XCU Section 2.8.2, "Exit status for commands" [20] "Secure programmer: Call components safely"
5.9 Comparison of padding data [padcomp]	CERT C Secure Coding Standard, EXP04-C. Do not perform byte-by-byte comparisons involving a structure[5]	[9] Chapter 6, "Structures" ("Structures and functions," p. 129) [13] Chapter 6, "C Language Issues" ("Structure padding," 284–287) [21] Question 2.8, Question 2.12
5.10 Converting a pointer to integer or integer to pointer [intptrconv]	CERT C Secure Coding Standard, INT11-C. Take care when converting from pointer to integer or integer to pointer[5] ISO/IEC TR 24772, "Pointer Casting and Pointer Type Changes [HFC]"[6] MITRE CWE: CWE-466: Return of Pointer Value outside of Expected Range CWE-587: Assignment of a Fixed Address to a Pointer[8]	n/a
5.11 Converting pointer values to more strictly aligned pointer types [alignconv]	CERT C Secure Coding Standard, EXP36-C. Do not convert pointers into more strictly aligned pointer types[5] ISO/IEC TR 24772, "Pointer Casting and Pointer Type Changes [HFC]"[6] MISRA-C 2004, Rules 11.2 and 11.3[7]	[22] *Computer Systems: A Programmer's Perspective*
5.12 Copying a `FILE` object [filecpy]	CERT C Secure Coding Standard, FIO38-C. Do not use a copy of a FILE object for input and output[5]	n/a
5.13 Declaring the same function or object in incompatible ways [funcdecl]	CERT C Secure Coding Standard, DCL40-C. Incompatible declarations of the same function or object[5]	[17] Section 2.8.3
5.14 Dereferencing an out-of-domain pointer [nullref]	CERT C Secure Coding Standard, EXP34-C. Do not dereference null pointers[5] ISO/IEC TR 24772, "Pointer Casting and Pointer Type Changes [HFC]" and "Null Pointer Dereference [XYH]"[6] MITRE CWE, CWE-476: NULL Pointer Dereference[8]	[11] Section 5.2.18, "Null-pointer dereference" [23] Vector Rewrite Attack [24] Unusualbugs

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Rule	Related Guidelines	References
5.15 Escaping of the address of an automatic object [addrescape]	CERT C Secure Coding Standard, DCL30-C. Declare objects with appropriate storage durations[5] ISO/IEC TR 24772, "Dangling References to Stack Frames [DCM]"[6] MISRA-C 2004, Rule 8.6[7]	[25] Coverity Prevent User's Manual (3.3.0)
5.16 Conversion of signed characters to wider integer types before a check for `EOF` [signconv]	CERT C Secure Coding Standard, STR34-C. Cast characters to unsigned char before converting to larger integer sizes[5] MISRA-C 2004, Rule 6.1[7] MITRE CWE, CWE-704: Incorrect Type Conversion or Cast[8]	n/a
5.17 Use of an implied default in a `switch` statement [swtchdflt]	CERT C Secure Coding Standard, MSC01-C. Strive for logical completeness[5] ISO/IEC TR 24772, "Switch Statements and Static Analysis [CLL]"[6]	[11] Section 5.2.17, "Failure to account for default case in switch" [17] Section 2.7.2, "Errors of omission and addition"
5.18 Failing to close files or free dynamic memory when they are no longer needed [fileclose]	CERT C Secure Coding Standard, FIO42-C. Ensure files are properly closed when they are no longer needed[5] MITRE CWE: CWE-403: Exposure of File Descriptor to Unintended Control Sphere CWE-404: Improper Resource Shutdown or Release[8]	[13] Chapter 10, "UNIX Processes" ("File descriptor leaks," 582–587) [26] [27] Inheritance (Windows) [28]
5.19 Failing to detect and handle standard library errors [liberr]	CERT C Secure Coding Standard, FIO04-C. Detect and handle input and output errors[5] MITRE CWE, CWE-391: Unchecked Error Condition[8]	[3] Chapter 8, "File I/O" [29] Section 6, "I/O error checking"
5.20 Forming invalid pointers by library function [libptr]	n/a	n/a
5.21 Allocating insufficient memory [insufmem]	n/a	n/a
5.22 Forming or using out-of-bounds pointers or array subscripts [invptr]	CERT C Secure Coding Standard, ARR30-C. Do not form or use out of bounds pointers or array subscripts[5] ISO/IEC TR 24772, "Unchecked Array Indexing [XYZ]"[6] MITRE CWE: CWE-119: Improper Restriction of Operations within the Bounds of a Memory Buffer CWE-121: Stack-based Buffer Overflow CWE-122: Heap-based Buffer Overflow CWE-129: Improper Validation of Array Index CWE-788: Access of Memory Location after End of Buffer[8] CWE-805: Buffer Access with Incorrect Length Value	[3] Chapter 1, "Running with Scissors" [11] Section 5.2.13, "Unchecked array indexing" [30] [31] [32] [33] "CVE-2008-1517: Apple Mac OS X (XNU) Missing Array Index Validation"

© ISO/IEC 2013 – All rights reserved

Rule	Related Guidelines	References
5.23 Freeing memory multiple times [dblfree]	CERT C Secure Coding Standard, MEM31-C. Free dynamically allocated memory exactly once[5] ISO/IEC TR 24772, "Dangling Reference to Heap [XYK]" and "Memory Leak [XYL]"[6] MITRE CWE, CWE-415: Double Free[8]	[3] Chapter 4, "Dynamic Memory Management" [10] Double Free [11] "Doubly freeing memory" [34] [35]
5.24 Including tainted or out-of-domain input in a format string [usrfmt]	CERT C Secure Coding Standard, FIO30-C. Exclude user input from format strings[5] ISO/IEC TR 24772, "Injection [RST]"[6] MITRE CWE, CWE-134: Uncontrolled Format String[8]	[3] Chapter 6, "Formatted Output" [11] Section 5.2.23, "Format string problem"
5.25 Incorrectly setting and using `errno` [inverrno]	CERT C Secure Coding Standard, ERR30-C. Set errno to zero before calling a library function known to set errno, and check errno only after the function returns a value indicating failure[5] MITRE CWE, CWE-456: Missing Initialization[8]	[36] Macros and Miscellaneous Pitfalls [37] Section 11, p. 168, and Section 14, p. 254 [38] Section 5.4, p. 73
5.26 Integer division errors [diverr]	CERT C Secure Coding Standard, INT33-C. Ensure that division and modulo operations do not result in divide-by-zero errors[5] MITRE CWE, CWE-369: Divide by Zero[8]	[3] Chapter 5, "Integer Security" [39] Chapter 2, "Basics"
5.27 Interleaving stream inputs and outputs without a flush or positioning call [ioileave]	CERT C Secure Coding Standard, FIO39-C. Do not alternately input and output from a stream without an intervening flush or positioning call[5]	n/a
5.28 Modifying string literals [strmod]	CERT C Secure Coding Standard, STR30-C. Do not attempt to modify string literals[5]	[21] comp.lang.c FAQ list, Question 1.32 [40] Topic 1.26, "strings—string literals"
5.29 Modifying the string returned by `getenv`, `localeconv`, `setlocale`, and `strerror` [libmod]	CERT C Secure Coding Standard, ENV30-C. Do not modify the object referenced by the return value of certain functions[5]	[14] getenv
5.30 Overflowing signed integers [intoflow]	CERT C Secure Coding Standard, INT32-C. Ensure that operations on signed integers do not result in overflow[5] MITRE CWE, CWE-190: Integer Overflow or Wraparound[8]	[3] Chapter 5, "Integer Security" [11] Section 5.2.7, "Integer overflow" [13] Chapter 6, "C Language Issues" ("Arithmetic boundary conditions," 211–223) [39] Chapter 2, "Basics" [41]
5.31 Passing a non-null-terminated character sequence to a library function [nonnullcs]	CERT C Secure Coding Standard, STR32-C. Null-terminate byte strings as required[5] ISO/IEC TR 24772, "String Termination [CJM]"[6] MITRE CWE: CWE-170: Improper Null Termination[8]	[3] Chapter 2, "Strings" [7] Section 5.2.14, "Miscalculated NULL termination" [42]
5.32 Passing arguments to character-handling functions that are not representable as `unsigned char` [chrsgnext]	CERT C Secure Coding Standard, STR37-C. Arguments to character handling functions must be representable as an unsigned char[5] MITRE CWE: CWE-686: Function Call with Incorrect Argument Type CWE-704: Incorrect Type Conversion or Cast[8]	[29] Section 1.1, "<ctype.h> and characters types"

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Rule	Related Guidelines	References
5.33 Passing pointers into the same object as arguments to different restrict-qualified parameters [restrict]	CERT C Secure Coding Standard, DCL33-C. Ensure that restrict-qualified source and destination pointers in function arguments do not reference overlapping objects[5] ISO/IEC TR 24772, "Passing Parameters and Return Values [CSJ]"[6]	n/a
5.34 Reallocating or freeing memory that was not dynamically allocated [xfree]	CERT C Secure Coding Standard, MEM34-C. Only free memory allocated dynamically[5] ISO/IEC TR 24772, "Type-breaking Reinterpretation of Data [AMV]"[6] MITRE CWE: CWE-590: Free of Memory Not on the Heap CWE-628: Function Call with Incorrectly Specified Arguments[8]	[43] Section 7.22.3.3, "The free function," Section 7.22.3.5, "The realloc function"
5.35 Referencing uninitialized memory [uninitref]	CERT C Secure Coding Standard: EXP33-C. Do not reference uninitialized memory MEM09-C. Do not assume memory allocation routines initialize memory[5] ISO/IEC TR 24772, "Initialization of Variables [LAV]"[6]	[44] [45]
5.36 Subtracting or comparing two pointers that do not refer to the same array [ptrobj]	CERT C Secure Coding Standard, ARR36-C. Do not subtract or compare two pointers that do not refer to the same array[5] MITRE CWE, CWE-469: Use of Pointer Subtraction to Determine Size[8]	[46] Section 5.3, "Pointers," and Section 5.7, "Expressions involving pointers"
5.37 Tainted strings are passed to a string copying function [taintstrcpy]	n/a	n/a
5.38 Taking the size of a pointer to determine the size of the pointed-to type [sizeofptr]	CERT C Secure Coding Standard, EXP01-C. Do not take the size of a pointer to determine the size of the pointed-to type[5] MITRE CWE, CWE-467: Use of sizeof() on a Pointer Type[8]	[11] Section 5.6.8, "Use of sizeof on a pointer type" [47] Section 2.1.1, "Respecting memory bounds"
5.39 Using a tainted value as an argument to an unprototyped function pointer [taintnoproto]	n/a	n/a
5.40 Using a tainted value to write to an object using a formatted input or output function [taintformatio]	CERT C Secure Coding Standard, STR31-C. Guarantee that storage for strings has sufficient space for character data and the null terminator[5]	n/a
5.41 Using a value for `fsetpos` other than a value returned from `fgetpos` [xfilepos]	CERT C Secure Coding Standard, FIO44-C. Only use values for fsetpos() that are returned from fgetpos()[5]	n/a
5.42 Using an object overwritten by `getenv`, `localeconv`, `setlocale`, and `strerror` [libuse]	CERT C Secure Coding Standard, ENV00-C. Do not store the pointer to the string returned by getenv()[5] ISO/IEC TR 24731-2[48]	[14] Chapter 8 and "Environment variables," strdup [27] _dupenv_s and _wdupenv_s, getenv_s, _wgetenv_s [49] Section 3.6, "Using environment variables securely"
5.43 Using character values that are indistinguishable from `EOF` [chreof]	CERT C Secure Coding Standard, FIO34-C. Use int to capture the return value of character IO functions[5]	[50] SAMATE Reference Dataset Test Case ID 000-000-088

© ISO/IEC 2013 – All rights reserved

Rule	Related Guidelines	References
5.44 Using identifiers that are reserved for the implementation [resident]	n/a	[26] Section 2.2, "The compilation environment" [43] Section 7.1.3, "Reserved identifiers"
5.45 Using invalid format strings [invfmtstr]	CERT C Secure Coding Standard, FIO00-C. Take care when creating format strings[5] MITRE CWE, CWE-686: Function Call with Incorrect Argument Type[8]	n/a
5.46 Tainted, potentially mutilated, or out-of-domain integer values are used in a restricted sink [taintsink]	CERT C Secure Coding Standard: ARR32-C. Ensure size arguments for variable length arrays are in a valid range INT04-C. Enforce limits on integer values originating from untrusted sources[5] ISO/IEC TR 24772, "Unchecked Array Indexing [XYZ]"[6]	[3] Chapter 5, "Integer Security" [51]

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Annex D
(informative)

Decidability of Rules

It is possible to implement static analysis for decidable rules in such a way that all occurrences of violations are diagnosed, with minimal or no false positives. Undecidable rules require static analysis implementations to choose a balance between false positives and false negatives.

5.1 [ptrcomp]	Undecidable		5.24 [usrfmt]	Undecidable
5.2 [accfree]	Undecidable		5.25 [inverrno]	Undecidable
5.3 [accsig]	Undecidable		5.26 [diverr]	Undecidable
5.4 [boolasgn]	Decidable		5.27 [ioileave]	Undecidable
5.5 [asyncsig]	Undecidable		5.28 [strmod]	Undecidable
5.6 [argcomp]	Undecidable		5.29 [libmod]	Undecidable
5.7 [sigcall]	Undecidable		5.30 [intoflow]	Undecidable
5.8 [syscall]	Undecidable		5.31 [nonnullcs]	Undecidable
5.9 [padcomp]	Undecidable		5.32 [chrsgnext]	Undecidable
5.10 [intptrconv]	Undecidable		5.33 [restrict]	Undecidable
5.11 [alignconv]	Undecidable		5.34 [xfree]	Undecidable
5.12 [filecpy]	Undecidable		5.35 [uninitref]	Undecidable
5.13 [funcdecl]	Decidable		5.36 [ptrobj]	Undecidable
5.14 [nullref]	Undecidable		5.37 [taintstrcpy]	Undecidable
5.15 [addrescape]	Undecidable		5.38 [sizeofptr]	Decidable
5.16 [signconv]	Undecidable		5.39 [taintnoproto]	Undecidable
5.17 [swtchdflt]	Decidable		5.40 [taintformatio]	Undecidable
5.18 [fileclose]	Undecidable		5.41 [xfilepos]	Undecidable
5.19 [liberr]	Undecidable		5.42 [libuse]	Undecidable
5.20 [libptr]	Undecidable		5.43 [chreof]	Undecidable
5.21 [insufmem]	Undecidable		5.44 [resident]	Decidable
5.22 [invptr]	Undecidable		5.45 [invfmtstr]	Undecidable
5.23 [dblfree]	Undecidable		5.46 [taintsink]	Undecidable

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Bibliography

[1] Chess B., & West J. *Secure Programming with Static Analysis*. Addison-Wesley, Boston, 2007

[2] ISO/IEC 11889-1:2009, *Information technology — Trusted Platform Module — Part 1: Overview*

[3] Seacord R.C. *Secure Coding in C and C++*. Addison-Wesley, Boston, 2013 (see http://www.cert.org/books/secure-coding for news and errata)

[4] ISO/IEC/IEEE 9945:2009, *Information technology — Portable Operating System Interface (POSIX®) Base Specifications, Issue 7*

[5] CERT C Secure Coding Standard https://www.securecoding.cert.org/confluence/x/HQE (2010)

[6] ISO/IEC TR 24772, *Information technology — Programming languages — Guidance to avoiding vulnerabilities in programming languages through language selection and use*

[7] Motor Industry Software Reliability Association. *MISRA-C 2004: Guidelines for the Use of the C Language in Critical Systems*. MISRA, 2004

[8] MITRE. Common Weakness Enumeration, Draft 9, April 2008

[9] Kernighan B.W., & Ritchie D.M. *The C Programming Language*. Prentice-Hall, Englewood Cliffs, NJ, Second Edition, 1988

[10] OWASP Foundation3. Open Web Application Security Project, 2011

[11] Viega J. CLASP Reference Guide, Volume 1.1. Secure Software, 2005

[12] *Rationale for international standard — Programming languages — C*, Revision 5.10, April 2003

[13] Dowd M., McDonald J., Schuh J. *The Art of Software Security Assessment: Identifying and Preventing Software Vulnerabilities*. Addison-Wesley, Boston, 2006

[14] The Open Group & the IEEE. *The Open Group Base Specifications, Issue 6 (IEEE Std 1003.1)*. Institute of Electrical and Electronics Engineers, New York, 2004

[15] OpenBSD, Berkley Software Design, Inc. Manual Pages, June 2008

[16] Zalewski M. Delivering Signals for Fun and Profit: Understanding, exploiting and preventing signal-handling related vulnerabilities, May 2001

[17] Hatton L. *Safer C: Developing Software for High-Integrity and Safety-Critical Systems*. McGraw-Hill Book Company, New York, 1995

[18] The MITRE Corporation. Common Vulnerabilities and Exposures List. http://cve.mitre.org/cve/cve.html (2011)

[19] Spinellis D. Code Quality: The Open Source Perspective. Addison-Wesley, Boston, 2006.

[20] Wheeler D. Secure programmer: Call components safely, December 2004

[21] Summit S. *C Programming FAQs: Frequently Asked Questions*. Addison-Wesley, Boston, 1995

[22] Bryant R.E., & O'Halloran D. *Computer Systems: A Programmer's Perspective*. Prentice Hall, Upper Saddle River, NJ, 2003

[23] Jack B. *Vector Rewrite Attack*. http://cansecwest.com/slides07/Vector-Rewrite-Attack.pdf (May 2007)

[24] van Sprundel I. Unusualbugs, 2006

 © ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

[25] Coverity Prevent User's Manual (3.3.0), 2007

[26] Institute of Electrical and Electronics Engineers (IEEE). The Open Group Base Specifications Issue 7 (IEEE Std 1003.1:2008). IEEE, New York, 2008

[27] Microsoft Developer Network. http://msdn.microsoft.com/en-us/ms348103 (2011)

[28] Network Associates Inc. (NAI). Bugtraq: Network Associates Inc. Advisory (OpenBSD), 1998

[29] Kettlewell R. C Language Gotchas. http://www.greenend.org.uk/rjk/2001/02/cfu.html (February 2002)

[30] Finlay I.A. CERT Advisory CA-2003-16, Buffer Overflow in Microsoft RPC. http://www.cert.org/advisories/CA-2003-16.html (July 2003)

[31] Microsoft Security Bulletin MS03-026, "Buffer Overrun In RPC Interface Could Allow Code Execution (823980)," September 2003

[32] Pethia R.D. Viruses and Worms: What Can We Do About Them?" September 10, 2003

[33] xorl %eax, %eax. CVE-2008-1517: Apple Mac OS X (XNU) Missing Array Index Validation, June 9, 2009

[34] MIT. MIT krb5 Security Advisory 2005-003," 2005

[35] Mead R. Vulnerability Note VU#623332, MIT Kerberos 5 contains double free vulnerability in "krb5_recvauth()" function, 2005

[36] Brainbell.com. Advice & Warnings for C Tutorials. http://www.brainbell.com/tutors/c/Advice_and_Warnings_for_C/ (2011)

[37] Horton M.R. *Portable C Software*. Prentice-Hall, Upper Saddle River, NJ, 1990

[38] Koenig A. *C Traps and Pitfalls*. Addison-Wesley Professional, Reading, MA, 1989

[39] Warren H.S. Hacker's Delight. Addison-Wesley, Boston, 2002

[40] Plum T. *C++ Programming*. Plum Hall, Kamuela, HI, 1991

[41] Giobbi R. Vulnerability Note VU#551436, *Mozilla Firefox SVG viewer vulnerable to buffer overflow,* 2007

[42] Schwarz B., Wagner Hao Chen, Morrison D., West G., Lin J., Tu J. Wei. Model Checking an Entire Linux Distribution for Security Violations." *Proceedings of the 21st Annual Computer Security Applications Conference*, December 2005

[43] ISO/IEC 9899:2011, *Information technology — Programming languages — C*

[44] Flake H. Attacks on Uninitialized Local Variables." http://www.blackhat.com/presentations/bh-europe-06/bh-eu-06-Flake.pdf (2006)

[45] Mercy. Exploiting Uninitialized Data, http://www.felinemenace.org/~mercy/papers/UBehavior/UBehavior.zip (January 2006)

[46] Banahan M., Brady D., Doran M. *The C Book, Featuring the ANSI C Standard*. Addison-Wesley, Boston, 1991

[47] Drepper U. Defensive Programming for Red Hat Enterprise Linux (and What To Do If Something Goes Wrong). http://web.sunybroome.edu/~antonakos_j/cst203/buffer/defprogramming.pdf (April 8, 2009)

[48] ISO/IEC TR 24731, *Extensions to the C Library—Part II: Dynamic Allocation Functions*

© ISO/IEC 2013 – All rights reserved

[49] Viega J., & Messier M. *Secure Programming Cookbook for C and C++: Recipes for Cryptography, Authentication, Networking, Input Validation & More*. O'Reilly, Sebastopol, CA, 2003

[50] National Institute of Standards and Technology (NIST). SAMATE Reference Dataset, 2006

[51] Griffiths A. Clutching at Straws: When You Can Shift the Stack Pointer." http://arsouyes.org/index.php?id=248 (2006)

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

ICS 35.060
Price based on 80 pages

© ISO/IEC 2013 – All rights reserved

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

This page deliberately left blank

NO COPYING WITHOUT BSI PERMISSION EXCEPT AS PERMITTED BY COPYRIGHT LAW

Licenced to TORINO POLITECNICO , CRUI Conferenza dei Rettori delle downloaded on 27/10/2024. For status of this document please check https://knowledge.bsigroup.com. Reproduction and distribution is prohibited.

British Standards Institution (BSI)

BSI is the national body responsible for preparing British Standards and other standards-related publications, information and services.

BSI is incorporated by Royal Charter. British Standards and other standardization products are published by BSI Standards Limited.

About us

We bring together business, industry, government, consumers, innovators and others to shape their combined experience and expertise into standards -based solutions.

The knowledge embodied in our standards has been carefully assembled in a dependable format and refined through our open consultation process. Organizations of all sizes and across all sectors choose standards to help them achieve their goals.

Information on standards

We can provide you with the knowledge that your organization needs to succeed. Find out more about British Standards by visiting our website at bsigroup.com/standards or contacting our Customer Services team or Knowledge Centre.

Buying standards

You can buy and download PDF versions of BSI publications, including British and adopted European and international standards, through our website at bsigroup.com/shop, where hard copies can also be purchased.

If you need international and foreign standards from other Standards Development Organizations, hard copies can be ordered from our Customer Services team.

Copyright in BSI publications

All the content in BSI publications, including British Standards, is the property of and copyrighted by BSI or some person or entity that owns copyright in the information used (such as the international standardization bodies) and has formally licensed such information to BSI for commercial publication and use.

Save for the provisions below, you may not transfer, share or disseminate any portion of the standard to any other person. You may not adapt, distribute, commercially exploit, or publicly display the standard or any portion thereof in any manner whatsoever without BSI's prior written consent.

Storing and using standards

Standards purchased in soft copy format:

- A British Standard purchased in soft copy format is licensed to a sole named user for personal or internal company use only.
- The standard may be stored on more than 1 device provided that it is accessible by the sole named user only and that only 1 copy is accessed at any one time.
- A single paper copy may be printed for personal or internal company use only.

Standards purchased in hard copy format:

- A British Standard purchased in hard copy format is for personal or internal company use only.
- It may not be further reproduced – in any format – to create an additional copy. This includes scanning of the document.

If you need more than 1 copy of the document, or if you wish to share the document on an internal network, you can save money by choosing a subscription product (see 'Subscriptions').

Reproducing extracts

For permission to reproduce content from BSI publications contact the BSI Copyright & Licensing team.

Subscriptions

Our range of subscription services are designed to make using standards easier for you. For further information on our subscription products go to bsigroup.com/subscriptions.

With **British Standards Online (BSOL)** you'll have instant access to over 55,000 British and adopted European and international standards from your desktop. It's available 24/7 and is refreshed daily so you'll always be up to date.

You can keep in touch with standards developments and receive substantial discounts on the purchase price of standards, both in single copy and subscription format, by becoming a **BSI Subscribing Member**.

PLUS is an updating service exclusive to BSI Subscribing Members. You will automatically receive the latest hard copy of your standards when they're revised or replaced.

To find out more about becoming a BSI Subscribing Member and the benefits of membership, please visit bsigroup.com/shop.

With a **Multi-User Network Licence (MUNL)** you are able to host standards publications on your intranet. Licences can cover as few or as many users as you wish. With updates supplied as soon as they're available, you can be sure your documentation is current. For further information, email subscriptions@bsigroup.com.

Revisions

Our British Standards and other publications are updated by amendment or revision.

We continually improve the quality of our products and services to benefit your business. If you find an inaccuracy or ambiguity within a British Standard or other BSI publication please inform the Knowledge Centre.

Useful Contacts

Customer Services
Tel: +44 345 086 9001
Email (orders): orders@bsigroup.com
Email (enquiries): cservices@bsigroup.com

Subscriptions
Tel: +44 345 086 9001
Email: subscriptions@bsigroup.com

Knowledge Centre
Tel: +44 20 8996 7004
Email: knowledgecentre@bsigroup.com

Copyright & Licensing
Tel: +44 20 8996 7070
Email: copyright@bsigroup.com

BSI Group Headquarters

389 Chiswick High Road London W4 4AL UK

bsi.